

Aborting a Connection by Canceling the Context

Another advantage to using a context is the cancel function itself. You can use it to cancel the connection attempt on demand, without specifying a deadline, as shown in Listing 3-7.

```
package ch03

import (
    "context"
    "net"
    "syscall"
    "testing"
    "time"
)

func TestDialContextCancel(t *testing.T) {
    ctx, cancel := ❶ context.WithCancel(context.Background())
    sync := make(chan struct{})

    ❷ go func() {
        defer func() { sync <- struct{}{} }()

        var d net.Dialer
        d.Control = func(_, _ string, _ syscall.RawConn) error {
            time.Sleep(time.Second)
            return nil
        }
        conn, err := d.DialContext(ctx, "tcp", "10.0.0.1:80")
        if err != nil {
            t.Log(err)
            return
        }

        conn.Close()
        t.Error("connection did not time out")
    }()

    ❸ cancel()
    <-sync

    if ctx.Err() != ❹ context.Canceled {
        t.Errorf("expected canceled context; actual: %q", ctx.Err())
    }
}
```

Listing 3-7: Directly canceling the context to abort the connection attempt (dial_cancel_test.go)

Instead of creating a context with a deadline and waiting for the deadline to abort the connection attempt, you use `context.WithCancel` to return a context and a function to cancel the context ❶. Since you're manually canceling the context, you create a closure and spin it off in a goroutine to handle the connection attempt ❷. Once the dialer is attempting to connect to and handshake with the remote node, you call the cancel function ❸

to cancel the context. This causes the `DialContext` method to immediately return with a non-nil error, exiting the goroutine. You can check the context's `Err` method to make sure the call to cancel was what resulted in the canceled context, as opposed to a deadline in Listing 3-6. In this case, the context's `Err` method should return a `context.Canceled` error ❹.

Canceling Multiple Dialers

You can pass the same context to multiple `DialContext` calls and cancel all the calls at the same time by executing the context's cancel function. For example, let's assume you need to retrieve a resource via TCP that is on several servers. You can asynchronously dial each server, passing each dialer the same context. You can then abort the remaining dialers after you receive a response from one of the servers.

In Listing 3-8, you pass the same context to multiple dialers. When you receive the first response, you cancel the context and abort the remaining dialers.

```
package ch03

import (
    "context"
    "net"
    "sync"
    "testing"
    "time"
)

func TestDialContextCancelFanOut(t *testing.T) {
    ❶ ctx, cancel := context.WithDeadline(
        context.Background(),
        time.Now().Add(10*time.Second),
    )

    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }
    defer listener.Close()

    ❷ go func() {
        // Only accepting a single connection.
        conn, err := listener.Accept()
        if err == nil {
            conn.Close()
        }
    }()

    ❸ dial := func(ctx context.Context, address string, response chan int,
        id int, wg *sync.WaitGroup) {
        defer wg.Done()

        var d net.Dialer
```

```

        c, err := d.DialContext(ctx, "tcp", address)
        if err != nil {
            return
        }
        c.Close()

        select {
        case <-ctx.Done():
        case response <- id:
        }
    }

    res := make(chan int)
    var wg sync.WaitGroup

    ❷ for i := 0; i < 10; i++ {
        wg.Add(1)
        go dial(ctx, listener.Addr().String(), res, i+1, &wg)
    }

    ❸ response := <-res
    cancel()
    wg.Wait()
    close(res)

    if ctx.Err() != ❹ context.Canceled {
        t.Errorf("expected canceled context; actual: %s",
            ctx.Err(),
        )
    }

    t.Logf("dialer %d retrieved the resource", response)
}

```

Listing 3-8: Canceling all outstanding dialers after receiving the first response (dial_fanout_test.go)

You create a context by using `context.WithDeadline` ❶ because you want to have three potential results when checking the context's `Err` method: `context.Canceled`, `context.DeadlineExceeded`, or `nil`. You expect `Err` will return the `context.Canceled` error, since your test aborts the dialers with a call to the `cancel` function.

First, you need a listener. This listener accepts a single connection and closes it after the successful handshake ❷. Next, you create your dialers. Since you're spinning up multiple dialers, it makes sense to abstract the dialing code to its own function ❸. This anonymous function dials out to the given address by using `DialContext`. If it succeeds, it sends the dialer's ID across the response channel, provided you haven't yet canceled the context. You spin up multiple dialers by calling `dial` in separate goroutines using a `for` loop ❹. If `dial` blocks on the call to `DialContext` because another dialer won the race, you cancel the context, either by way of the `cancel` function or the deadline, causing the `dial` function to exit early. You use a wait group to make sure the test doesn't proceed until all `dial` goroutines terminate after you cancel the context.

Once the goroutines are running, one will win the race and make a successful connection to the listener. You receive the winning dialer's ID on the `res` channel ❹, then abort the losing dialers by canceling the context. At this point, the call to `wg.Wait` blocks until the aborted dialer goroutines return. Finally, you make sure it was your call to cancel that caused the cancellation of the context ❺. Calling `cancel` does not guarantee that `Err` will return `context.Canceled`. The deadline can cancel the context, at which point calls to `cancel` become a no-op and `Err` will return `context.DeadlineExceeded`. In practice, the distinction may not matter to you, but it's there if you need it.

Implementing Deadlines

Go's network connection objects allow you to include deadlines for both read and write operations. Deadlines allow you to control how long network connections can remain idle, where no packets traverse the connection. You can control the Read deadline by using the `SetReadDeadline` method on the connection object, control the Write deadline by using the `SetWriteDeadline` method, or both by using the `SetDeadline` method. When a connection reaches its read deadline, all currently blocked and future calls to a network connection's `Read` method immediately return a time-out error. Likewise, a network connection's `Write` method returns a time-out error when the connection reaches its write deadline.

Go's network connections don't set any deadline for reading and writing operations by default, meaning your network connections may remain idle for a long time. This could prevent you from detecting network failures, like an unplugged cable, in a timely manner, because it's tougher to detect network issues between two nodes when no traffic is in flight.

The server in Listing 3-9 implements a deadline on its connection object.

```
package ch03

import (
    "io"
    "net"
    "testing"
    "time"
)

func TestDeadline(t *testing.T) {
    sync := make(chan struct{})

    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    go func() {
        conn, err := listener.Accept()
        if err != nil {
            t.Log(err)
            return
        }
    }()

    sync.WaitGroup().Add(1)
    defer sync.Done()

    conn.SetDeadline(time.Now().Add(100 * time.Millisecond))
    _, err = conn.Read(make([]byte, 1024))
    if err != nil {
        t.Log(err)
    }
}
```

```

    }
    defer func() {
        conn.Close()
        close(sync) // read from sync shouldn't block due to early return
    }()

    ❶ err = conn.SetDeadline(time.Now().Add(5 * time.Second))
    if err != nil {
        t.Error(err)
        return
    }

    buf := make([]byte, 1)
    _, err = conn.Read(buf) // blocked until remote node sends data
    nErr, ok := err.(net.Error)
    if !ok || ❷!nErr.Timeout() {
        t.Errorf("expected timeout error; actual: %v", err)
    }

    sync <- struct{}{}

    ❸ err = conn.SetDeadline(time.Now().Add(5 * time.Second))
    if err != nil {
        t.Error(err)
        return
    }

    _, err = conn.Read(buf)
    if err != nil {
        t.Error(err)
    }
}()

conn, err := net.Dial("tcp", listener.Addr().String())
if err != nil {
    t.Fatal(err)
}
defer conn.Close()

<-sync
_, err = conn.Write([]byte("1"))
if err != nil {
    t.Fatal(err)
}

buf := make([]byte, 1)
_, err = conn.Read(buf) // blocked until remote node sends data
if err != ❹io.EOF {
    t.Errorf("expected server termination; actual: %v", err)
}
}

```

Listing 3-9: A server-enforced deadline terminates the network connection (deadline_test.go).

Once the server accepts the client's TCP connection, you set the connection's read deadline ❶. Since the client won't send data, the call to `Read` will block until the connection exceeds the read deadline. After five seconds, `Read` returns an error, which you verify is a time-out ❷. Any future reads to the connection object will immediately result in another time-out error. However, you can restore the functionality of the connection object by pushing the deadline forward again ❸. After you've done this, a second call to `Read` succeeds. The server closes its end of the network connection, which initiates the termination process with the client. The client, currently blocked on its `Read` call, returns `io.EOF` ❹ when the network connection closes.

We typically use deadlines to provide a window of time during which the remote node can send data over the network connection. When you read data from the remote node, you push the deadline forward. The remote node sends more data, and you push the deadline forward again, and so on. If you don't hear from the remote node in the allotted time, you can assume that either the remote node is gone and you never received its FIN or that it is idle.

Implementing a Heartbeat

For long-running network connections that may experience extended idle periods at the application level, it's wise to implement a heartbeat between nodes to advance the deadline. This allows you to quickly identify network issues and promptly reestablish a connection as opposed to waiting to detect the network error when your application goes to transmit data. In this way, you can help make sure your application always has a good network connection when it needs it.

For our purposes, a *heartbeat* is a message sent to the remote side with the intention of eliciting a reply, which we can use to advance the deadline of our network connection. Nodes send these messages at a regular interval, like a heartbeat. Not only is this method portable over various operating systems, but it also makes sure the application using the network connection is responding, since the application implements the heartbeat. Also, this technique tends to play well with firewalls that may block TCP keepalives. We'll discuss keepalives in Chapter 4.

To start, you'll need a bit of code you can run in a goroutine to ping at regular intervals. You don't want to needlessly ping the remote node when you recently received data from it, so you need a way to reset the ping timer. Listing 3-10 is a simple implementation from a file named *ping.go* that meets those requirements.

I use *ping* and *pong* messages in my heartbeat examples, where the reception of a ping message—the challenge—tells the receiver it should reply with a pong message—the response. The challenge and response messages are arbitrary. You could use anything you want to here, provided the remote node knows your intention is to elicit its reply.

```
package ch03
```

```
import (
```

```

    "context"
    "io"
    "time"
)

const defaultPingInterval = 30 * time.Second

func Pinger(ctx context.Context, w io.Writer, reset <-chan time.Duration) {
    var interval time.Duration
    select {
        case <-ctx.Done():
            return
        ❶ case interval = <-reset: // pulled initial interval off reset channel
            default:
            }
        if interval <= 0 {
            interval = defaultPingInterval
        }

        ❷ timer := time.NewTimer(interval)
        defer func() {
            if !timer.Stop() {
                <-timer.C
            }
        }()

        for {
            select {
                ❸ case <-ctx.Done():
                    return
                ❹ case newInterval := <-reset:
                    if !timer.Stop() {
                        <-timer.C
                    }
                    if newInterval > 0 {
                        interval = newInterval
                    }
                ❺ case <-timer.C:
                    if _, err := w.Write([]byte("ping")); err != nil {
                        // track and act on consecutive timeouts here
                        return
                    }
            }

            ❻ _ = timer.Reset(interval)
        }
    }
}

```

Listing 3-10: A function that pings a network connection at a regular interval (ping.go)

The `Pinger` function writes ping messages to a given writer at regular intervals. Because it's meant to run in a goroutine, `Pinger` accepts a context as its first argument so you can terminate it and prevent it from leaking. Its remaining arguments include an `io.Writer` interface and a channel to signal a

timer reset. You create a buffered channel and put a duration on it to set the timer's initial interval ❶. If the interval isn't greater than zero, you use the default ping interval.

You initialize the timer to the interval ❷ and set up a deferred call to drain the timer's channel to avoid leaking it, if necessary. The endless for loop contains a select statement, where you block until one of three things happens: the context is canceled, a signal to reset the timer is received, or the timer expires. If the context is canceled ❸, the function returns, and no further pings will be sent. If the code selects the reset channel ❹, you shouldn't send a ping, and the timer resets ❺ before iterating on the select statement again.

If the timer expires ❻, you write a ping message to the writer, and the timer resets before the next iteration. If you wanted, you could use this case to keep track of any consecutive time-outs that occur while writing to the writer. To do this, you could pass in the context's cancel function and call it here if you reach a threshold of consecutive time-outs.

Listing 3-11 illustrates how to use the `Pinger` function introduced in Listing 3-10 by giving it a writer and running it in a goroutine. You can then read pings from the reader at the expected intervals and reset the ping timer with different intervals.

```
package ch03
```

```
import (
    "context"
    "fmt"
    "io"
    "time"
)

func ExamplePinger() {
    ctx, cancel := context.WithCancel(context.Background())
    r, w := io.Pipe() // in lieu of net.Conn
    done := make(chan struct{})
    ❶ resetTimer := make(chan time.Duration, 1)
    resetTimer <- time.Second // initial ping interval

    go func() {
        Pinger(ctx, w, resetTimer)
        close(done)
    }()

    receivePing := func(d time.Duration, r io.Reader) {
        if d >= 0 {
            fmt.Printf("resetting timer (%s)\n", d)
            resetTimer <- d
        }
    }

    now := time.Now()
    buf := make([]byte, 1024)
    n, err := r.Read(buf)
```



```

        if err != nil {
            fmt.Println(err)
        }

        fmt.Printf("received %q (%s)\n",
            buf[:n], time.Since(now).Round(100*time.Millisecond))
    }

❷ for i, v := range []int64{0, 200, 300, 0, -1, -1, -1} {
    fmt.Printf("Run %d:\n", i+1)
    receivePing(time.Duration(v)*time.Millisecond, r)
}

cancel()
<-done // ensures the pinger exits after canceling the context

// Output:
❸ // Run 1:
// resetting timer (0s)
// received "ping" (1s)
❹ // Run 2:
// resetting timer (200ms)
// received "ping" (200ms)
❺ // Run 3:
// resetting timer (300ms)
// received "ping" (300ms)
❻ // Run 4:
// resetting timer (0s)
// received "ping" (300ms)
❼ // Run 5:
// received "ping" (300ms)
// Run 6:
// received "ping" (300ms)
// Run 7:
// received "ping" (300ms)
}

```

Listing 3-11: Testing the pinger and resetting its ping timer interval (ping_example_test.go)

In this example, you create a buffered channel ❶ that you'll use to signal a reset of the Pinger's timer. You put an initial ping interval of one second on the `resetTimer` channel before passing the channel to the `Pinger` function. You'll use this duration to initialize the Pinger's timer and dictate when to write the ping message to the writer.

You run through a series of millisecond durations in a loop ❷, passing each to the `receivePing` function. This function resets the ping timer to the given duration and then waits to receive the ping message on the given reader. Finally, it prints to `stdout` the time it takes to receive the ping message. Go checks `stdout` against the expected output in the example.

During the first iteration ❸, you pass in a duration of zero, which tells the Pinger to reset its timer by using the previous duration—one second in this example. As expected, the reader receives the ping message after one

second. The second iteration ④ resets the ping timer to 200 ms. Once this expires, the reader receives the ping message. The third run resets the ping timer to 300 ms ⑤, and the ping arrives at the 300 ms mark.

You pass in a zero duration for run 4 ⑥, preserving the 300 ms ping timer from the previous run. I find the technique of using zero durations to mean “use the previous timer duration” useful because I do not need to keep track of the initial ping timer duration. I can simply initialize the timer with the duration I want to use for the remainder of the TCP session and reset the timer by passing in a zero duration every time I need to preempt the transmission of the next ping message. Changing the ping timer duration in the future involves the modification of a single line as opposed to every place I send on the `resetTimer` channel.

Runs 5 to 7 ⑦ simply listen for incoming pings without resetting the ping timer. As expected, the reader receives a ping at 300 ms intervals for the last three runs.

With Listing 3-10 saved to a file named *ping.go* and Listing 3-11 saved to a file named *ping_example_test.go*, you can run the example by executing the following:

```
$ go test ping.go ping_example_test.go
```

Advancing the Deadline by Using the Heartbeat

Each side of a network connection could use a `Pinger` to advance its deadline if the other side becomes idle, whereas the previous examples showed only a single side using a `Pinger`. When either node receives data on the network connection, its ping timer should reset to stop the delivery of an unnecessary ping. Listing 3-12 is a new file named *ping_test.go* that shows how you can use incoming messages to advance the deadline.

```
package ch03

import (
    "context"
    "io"
    "net"
    "testing"
    "time"
)

func TestPingerAdvanceDeadline(t *testing.T) {
    done := make(chan struct{})
    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    begin := time.Now()
    go func() {
        defer func() { close(done) }()

```

```

conn, err := listener.Accept()
if err != nil {
    t.Log(err)
    return
}
ctx, cancel := context.WithCancel(context.Background())
defer func() {
    cancel()
    conn.Close()
}()

resetTimer := make(chan time.Duration, 1)
resetTimer <- time.Second
go Pinger(ctx, conn, resetTimer)

err = conn.SetDeadline(time.Now().Add(❶5 * time.Second))
if err != nil {
    t.Error(err)
    return
}

buf := make([]byte, 1024)
for {
    n, err := conn.Read(buf)
    if err != nil {
        return
    }
    t.Logf("[%s] %s",
        time.Since(begin).Truncate(time.Second), buf[:n])

    ❷ resetTimer <- 0
    err = ❸conn.SetDeadline(time.Now().Add(5 * time.Second))
    if err != nil {
        t.Error(err)
        return
    }
}
}()

conn, err := net.Dial("tcp", listener.Addr().String())
if err != nil {
    t.Fatal(err)
}
defer conn.Close()

buf := make([]byte, 1024)
❹ for i := 0; i < 4; i++ { // read up to four pings
    n, err := conn.Read(buf)
    if err != nil {
        t.Fatal(err)
    }
    t.Logf("[%s] %s", time.Since(begin).Truncate(time.Second), buf[:n])
}

_, err = ❺conn.Write([]byte("PONG!!!")) // should reset the ping timer
if err != nil {

```

```

        t.Fatal(err)
    }
    ❸ for i := 0; i < 4; i++ { // read up to four more pings
        n, err := conn.Read(buf)
        if err != nil {
            if err != io.EOF {
                t.Fatal(err)
            }
            break
        }
        t.Logf("[%s] %s", time.Since(begin).Truncate(time.Second), buf[:n])
    }
    <-done
    end := time.Since(begin).Truncate(time.Second)
    t.Logf("[%s] done", end)
    if end != ❹9*time.Second {
        t.Fatalf("expected EOF at 9 seconds; actual %s", end)
    }
}

```

Listing 3-12: Receiving data advances the deadline (ping_test.go)

You start a listener that accepts a connection, spins off a `Pinger` set to ping every second, and sets the initial deadline to five seconds ❶. From a client's perspective, it receives four pings followed by an `io.EOF` when the server reaches its deadline and terminates its side of the connection. However, a client can advance the server's deadline by sending the server data ❷ before the server reaches its deadline.

If the server reads data from its connection, it can be confident the network connection is still good. Therefore, it can inform the `Pinger` to reset ❸ its timer and push the connection's deadline forward ❹. To preempt the termination of the socket, the client listens for four ping messages ❺ from the server before sending an emphatic pong message ❻. This should buy the client five more seconds until the server reaches its deadline. The client reads four more pings ❼ and then waits for the inevitable. You check that a total of nine seconds ❶ has elapsed by the time the server terminates the connection, indicating the client's pong successfully triggered the reset of the ping timer.

In practice, this method of advancing the ping timer cuts down on the consumption of bandwidth by unnecessary pings. There is rarely a need to challenge the remote side of a network connection if you just received data on the connection.

The strings "ping" and "pong" are arbitrary. You could use smaller payloads, such as a single byte, for the same purpose, provided both sides of the network connection agree upon what values constitute a ping and a pong.

What You’ve Learned

We covered a lot of ground in this chapter. We started with a dive into TCP’s handshake, sequences, and acknowledgments, the sliding window, and connection terminations. Then, we covered the process of establishing TCP connections using Go’s standard library. We talked about temporary errors, time-outs, listening for incoming connections, and dialing remote services. Finally, we covered techniques to help you detect and timely correct network integrity issues.

I strongly recommend picking up *Practical Packet Analysis* by Chris Sanders (No Starch Press, 2017) and installing Wireshark. Manipulating your network code and seeing how it affects TCP traffic in Wireshark is a fantastic way to gain a deeper understanding of both TCP and Go’s networking packages. The next chapter covers sending and receiving data over TCP connections. Wireshark will help you gain a deeper understanding of data you send, including each payload’s effects on the sliding window. Familiarizing yourself with it now will pay dividends.

4

SENDING TCP DATA



Now that you know how to properly establish and gracefully terminate TCP connections in Go, it's time to put that knowledge to use by transmitting data. This chapter covers various techniques for sending and receiving data over a network using TCP.

We'll talk about the most common methods of reading data from network connections. You'll create a simple messaging protocol that allows you to transmit dynamically sized payloads between nodes. You'll then explore the networking possibilities afforded by the `net.Conn` interface. The chapter concludes with a deeper dive into the `TCPConn` object and insidious TCP networking problems that Go developers may experience.

Using the `net.Conn` Interface

Most of the network code in this book uses Go's `net.Conn` interface whenever possible, because it provides the functionality we need for most cases. You can write powerful network code using the `net.Conn` interface without having to assert its underlying type, ensuring your code is compatible across operating systems and allowing you to write more robust tests. (You will learn how to access `net.Conn`'s underlying type to use its more advanced methods later in this chapter.) The methods available on `net.Conn` cover most use cases.

The two most useful `net.Conn` methods are `Read` and `Write`. These methods implement the `io.Reader` and `io.Writer` interfaces, respectively, which are ubiquitous in the Go standard library and ecosystem. As a result, you can leverage the vast amounts of code written for those interfaces to create incredibly powerful network applications.

You use `net.Conn`'s `Close` method to close the network connection. This method will return `nil` if the connection successfully closed or an error otherwise. The `SetReadDeadline` and `SetWriteDeadline` methods, which accept a `time.Time` object, set the absolute time after which reads and writes on the network connection will return an error. The `SetDeadline` method sets both the read and write deadlines at the same time. As discussed in “Implementing Deadlines” on page 62, deadlines allow you to control how long a network connection may remain idle and allow for timely detection of network connectivity problems.

Sending and Receiving Data

Reading data from a network connection and writing data to it is no different from reading and writing to a file object, since `net.Conn` implements the `io.ReadWriteCloser` interface used to read and write to files. In this section, you'll first learn how to read data into a fixed-size buffer. Next, you'll learn how to use `bufio.Scanner` to read data from a network connection until it encounters a specific delimiter. You'll then explore TLV, an encoding method that enables you to define a basic protocol to dynamically allocate buffers for varying payload sizes. Finally, you'll see how to handle errors when reading from and writing to network connections.

Reading Data into a Fixed Buffer

TCP connections in Go implement the `io.Reader` interface, which allows you to read data from the network connection. To read data from a network connection, you need to provide a buffer for the network connection's `Read` method to fill.

The `Read` method will populate the buffer to its capacity if there is enough data in the connection's receive buffer. If there are fewer bytes in the receive buffer than the capacity of the buffer you provide, `Read` will populate the given buffer with the data and return instead of waiting for more

data to arrive. In other words, `Read` is not guaranteed to fill your buffer to capacity before it returns. Listing 4-1 demonstrates the process of reading data from a network connection into a byte slice.

```
package main

import (
    "crypto/rand"
    "io"
    "net"
    "testing"
)

func TestReadIntoBuffer(t *testing.T) {
    ❶ payload := make([]byte, 1<<24) // 16 MB
    _, err := rand.Read(payload) // generate a random payload
    if err != nil {
        t.Fatal(err)
    }

    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    go func() {
        conn, err := listener.Accept()
        if err != nil {
            t.Log(err)
            return
        }
        defer conn.Close()

        ❷ _, err = conn.Write(payload)
        if err != nil {
            t.Error(err)
        }
    }()

    conn, err := net.Dial("tcp", listener.Addr().String())
    if err != nil {
        t.Fatal(err)
    }

    buf := make([]byte, ❸ 1<<19) // 512 KB

    for {
        ❹ n, err := conn.Read(buf)
        if err != nil {
            if err != io.EOF {
                t.Error(err)
            }
            break
        }
    }
}
```

```

    }

    t.Logf("read %d bytes", n) // buf[:n] is the data read from conn
}

conn.Close()
}

```

Listing 4-1: Receiving data over a network connection (read_test.go)

You need something for the client to read, so you create a 16MB payload of random data ❶—more data than the client can read in its chosen buffer size of 512KB ❸ so that it will make at least a few iterations around its for loop. It's perfectly acceptable to use a larger buffer or a smaller payload and read the entirety of the payload in a single call to `Read`. Go correctly processes the data regardless of the payload and receive buffer sizes.

You then spin up the listener and create a goroutine to listen for incoming connections. Once accepted, the server writes the entire payload to the network connection ❷. The client then reads up to the first 512KB from the connection ❹ before continuing around the loop. The client continues to read up to 512KB at a time until either an error occurs or the client reads the entire 16MB payload.

Delimited Reading by Using a Scanner

Reading data from a network connection by using the method I just showed means your code needs to make sense of the data it receives. Since TCP is a stream-oriented protocol, a client can receive a stream of bytes across many packets. Unlike sentences, binary data doesn't include inherent punctuation that tells you where one message starts and stops.

If, for example, your code is reading a series of email messages from a server, your code will have to inspect each byte for delimiters indicating the boundaries of each message in the stream of bytes. Alternatively, your client may have an established protocol with the server whereby the server sends a fixed number of bytes to indicate the payload size the server will send next. Your code can then use this size to create an appropriate buffer for the payload. You'll see an example of this technique a little later in this chapter.

However, if you choose to use a delimiter to indicate the end of one message and the beginning of another, writing code to handle edge cases isn't so simple. For example, you may read 1KB of data from a single `Read` on the network connection and find that it contains two delimiters. This indicates that you have two complete messages, but you don't have enough information about the chunk of data following the second delimiter to know whether it is also a complete message. If you read another 1KB of data and find no delimiters, you can conclude that this entire block of data is a continuation of the last message in the previous 1KB you read. But what if you read 1KB of nothing but delimiters?

If this is starting to sound a bit complex, it's because you must account for data across multiple `Read` calls and handle any errors along the way.

Anytime you're tempted to roll your own solution to such a problem, check the standard library to see if a tried-and-true implementation already exists. In this case, `bufio.Scanner` does what you need.

The `bufio.Scanner` is a convenient bit of code in Go's standard library that allows you to read delimited data. The `Scanner` accepts an `io.Reader` as its input. Since `net.Conn` has a `Read` method that implements the `io.Reader` interface, you can use the `Scanner` to easily read delimited data from a network connection. Listing 4-2 sets up a listener to serve up delimited data for later parsing by `bufio.Scanner`.

```
package main

import (
    "bufio"
    "net"
    "reflect"
    "testing"
)

const ❶ payload = "The bigger the interface, the weaker the abstraction."

func TestScanner(t *testing.T) {
    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    go func() {
        conn, err := listener.Accept()
        if err != nil {
            t.Error(err)
            return
        }
        defer conn.Close()

        _, err = conn.Write([]byte(payload))
        if err != nil {
            t.Error(err)
        }
    }()
}

--snip--
```

Listing 4-2: Creating a test to serve up a constant payload (scanner_test.go)

This listener should look familiar by now. All it's meant to do is serve up the payload ❶. Listing 4-3 uses `bufio.Scanner` to read a string from the network, splitting each chunk by whitespace.

```
--snip--

conn, err := net.Dial("tcp", listener.Addr().String())
if err != nil {
```

```

        t.Fatal(err)
    }
    defer conn.Close()

    ❶ scanner := bufio.NewScanner(conn)
    scanner.Split(bufio.ScanWords)

    var words []string

    ❷ for scanner.Scan() {
        words = append(words, ❸ scanner.Text())
    }

    err = scanner.Err()
    if err != nil {
        t.Error(err)
    }

    expected := []string{"The", "bigger", "the", "interface,", "the",
        "weaker", "the", "abstraction."}

    if !reflect.DeepEqual(words, expected) {
        t.Fatal("inaccurate scanned word list")
    }
    ❹ t.Logf("Scanned words: %#v", words)
}

```

Listing 4-3: Using `bufio.Scanner` to read whitespace-delimited text from the network (scanner_test.go)

Since you know you're reading a string from the server, you start by creating a `bufio.Scanner` that reads from the network connection ❶. By default, the scanner will split data read from the network connection when it encounters a newline character (`\n`) in the stream of data. Instead, you elect to have the scanner delimit the input at the end of each word by using `bufio.ScanWords`, which will split the data when it encounters a word border, such as whitespace or sentence-terminating punctuation.

You keep reading data from the scanner as long as it tells you it's read data from the connection ❷. Every call to `Scan` can result in multiple calls to the network connection's `Read` method until the scanner finds its delimiter or reads an error from the connection. It hides the complexity of searching for a delimiter across one or more reads from the network connection and returning the resulting messages.

The call to the scanner's `Text` method returns the chunk of data as a string—a single word and adjacent punctuation, in this case—that it just read from the network connection ❸. The code continues to iterate around the `for` loop until the scanner receives an `io.EOF` or other error from the network connection. If it's the latter, the scanner's `Err` method will return a non-nil error. You can view the scanned words ❹ by adding the `-v` flag to the `go test` command.

Dynamically Allocating the Buffer Size

You can read data of variable length from a network connection, provided that both the sender and receiver have agreed on a protocol for doing so. The *type-length-value (TLV)* encoding scheme is a good option. TLV encoding uses a fixed number of bytes to represent the type of data, a fixed number of bytes to represent the value size, and a variable number of bytes to represent the value itself. Our implementation uses a 5-byte header: 1 byte for the type and 4 bytes for the length. The TLV encoding scheme allows you to send a type as a series of bytes to a remote node and constitute the same type on the remote node from the series of bytes.

Listing 4-4 defines the types that our TLV encoding protocol will accept.

```
package main

import (
    "bytes"
    "encoding/binary"
    "errors"
    "fmt"
    "io"
)

const (
    ❶ BinaryType uint8 = iota + 1
    ❷ StringType

    ❸ MaxPayloadSize uint32 = 10 << 20 // 10 MB
)

var ErrMaxPayloadSize = errors.New("maximum payload size exceeded")

type ❹Payload interface {
    fmt.Stringer
    io.ReaderFrom
    io.WriterTo
    Bytes() []byte
}
```

Listing 4-4: The message struct implements a simple protocol (types.go).

You start by creating constants to represent each type you will define. In this example, you will create a `BinaryType` ❶ and a `StringType` ❷. After digesting the implementation details of each type, you should be able to create types that fit your needs. For security purposes that we'll discuss in just a moment, you must define a maximum payload size ❸.

You also define an interface named `Payload` ❹ that describes the methods each type must implement. Each type must have the following methods: `Bytes`, `String`, `ReadFrom`, and `WriteTo`. The `io.ReaderFrom` and `io.WriterTo` interfaces allow your types to read from readers and write to writers, respectively. You have some flexibility in this regard. You could just as easily make the

Payload implement the `encoding.BinaryMarshaler` interface to marshal itself to a byte slice and the `encoding.BinaryUnmarshaler` interface to unmarshal itself from a byte slice. But the byte slice is one level removed from the network connection, so you'll keep the `Payload` interface as is. Besides, you'll use the binary encoding interfaces in the next chapter.

You now have the foundation built to create TLV-based types. Listing 4-5 details the first type, `Binary`.

--snip--

```
type ❶Binary []byte

func (m Binary) ❷Bytes() []byte { return m }
func (m Binary) ❸String() string { return string(m) }

func (m Binary) ❹WriteTo(w io.Writer) (int64, error) {
    err := ❺binary.Write(w, binary.BigEndian, BinaryType) // 1-byte type
    if err != nil {
        return 0, err
    }
    var n int64 = 1

    err = ❻binary.Write(w, binary.BigEndian, uint32(len(m))) // 4-byte size
    if err != nil {
        return n, err
    }
    n += 4

    o, err := ❼w.Write(m) // payload

    return n + int64(o), err
}
```

Listing 4-5: Creating the Binary type (types.go)

The `Binary` type ❶ is a byte slice; therefore, its `Bytes` method ❷ simply returns itself. Its `String` method ❸ casts itself as a string before returning. The `WriteTo` method accepts an `io.Writer` and returns the number of bytes written to the writer and an error interface ❹. The `WriteTo` method first writes the 1-byte type to the writer ❺. It then writes the 4-byte length of the `Binary` to the writer ❻. Finally, it writes the `Binary` value itself ❼.

Listing 4-6 rounds out the `Binary` type with its `ReadFrom` method.

--snip--

```
func (m *Binary) ReadFrom(r io.Reader) (int64, error) {
    var typ uint8
    err := ❶binary.Read(r, binary.BigEndian, &typ) // 1-byte type
    if err != nil {
        return 0, err
    }
    var n int64 = 1
    if typ != ❷BinaryType {
```

```

        return n, errors.New("invalid Binary")
    }

    var size uint32
    err = ❸binary.Read(r, binary.BigEndian, &size) // 4-byte size
    if err != nil {
        return n, err
    }
    n += 4
    ❹ if size > MaxPayloadSize {
        return n, ErrMaxPayloadSize
    }

    ❺ *m = make([]byte, size)
    o, err := ❻r.Read(*m) // payload

    return n + int64(o), err
}

```

Listing 4-6: Completing the Binary type's implementation (types.go)

The `ReadFrom` method reads ❶ 1 byte from the reader into the `typ` variable. It next verifies ❷ that the type is `BinaryType` before proceeding. Then it reads ❸ the next 4 bytes into the `size` variable, which sizes the new `Binary` byte slice ❺. Finally, it populates the `Binary` byte slice ❻.

Notice that you enforce a maximum payload size ❹. This is because the 4-byte integer you use to designate the payload size has a maximum value of 4,294,967,295, indicating a payload of over 4GB. With such a large payload size, it would be easy for a malicious actor to perform a denial-of-service attack that exhausts all the available random access memory (RAM) on your computer. Keeping the maximum payload size reasonable makes memory exhaustion attacks harder to execute.

Listing 4-7 introduces the `String` type, which, like `Binary`, implements the `Payload` interface.

--snip--

```

type String string

func (m String) ❶Bytes() []byte { return []byte(m) }
func (m String) ❷String() string { return string(m) }

func (m String) ❸WriteTo(w io.Writer) (int64, error) {
    err := ❹binary.Write(w, binary.BigEndian, StringType) // 1-byte type
    if err != nil {
        return 0, err
    }
    var n int64 = 1

    err = binary.Write(w, binary.BigEndian, uint32(len(m))) // 4-byte size
    if err != nil {
        return n, err
    }
}

```

```

    n += 4

    o, err := ❸w.Write([]byte(m)) // payload

    return n + int64(o), err
}

```

Listing 4-7: Creating the String type (types.go)

The String implementation's Bytes method ❶ casts the String to a byte slice. The String method ❷ casts the String type to its base type, string. The String type's WriteTo method ❸ is like Binary's WriteTo method except the first byte written ❹ is the StringType and it casts the String to a byte slice before writing it to the writer ❺.

Listing 4-8 finishes up the String type's Payload implementation.

--snip--

```

func (m *String) ReadFrom(r io.Reader) (int64, error) {
    var typ uint8
    err := binary.Read(r, binary.BigEndian, &typ) // 1-byte type
    if err != nil {
        return 0, err
    }
    var n int64 = 1
    if typ != ❶StringType {
        return n, errors.New("invalid String")
    }

    var size uint32
    err = binary.Read(r, binary.BigEndian, &size) // 4-byte size
    if err != nil {
        return n, err
    }
    n += 4

    buf := make([]byte, size)
    o, err := r.Read(buf) // payload
    if err != nil {
        return n, err
    }
    ❷ *m = String(buf)

    return n + int64(o), nil
}

```

Listing 4-8: Completing the String type's implementation (types.go)

Here, too, String's ReadFrom method is like Binary's ReadFrom method, with two exceptions. First, the method compares the typ variable against the StringType ❶ before proceeding. Second, the method casts the value read from the reader to a String ❷.

All that's left to implement is a way to read arbitrary data from a network connection and use it to constitute one of our two types. For that, we turn to Listing 4-9.

--snip--

```
func ❶ decode(r io.Reader) (Payload, error) {
    var typ uint8
    err := ❷ binary.Read(r, binary.BigEndian, &typ)
    if err != nil {
        return nil, err
    }

    ❸ var payload Payload

    switch ❹ typ {
    case BinaryType:
        payload = new(Binary)
    case StringType:
        payload = new(String)
    default:
        return nil, errors.New("unknown type")
    }

    _, err = payload.ReadFrom(
        ❺ io.MultiReader(bytes.NewReader([]byte{typ}), r))
    if err != nil {
        return nil, err
    }

    return payload, nil
}
```

Listing 4-9: Decoding bytes from a reader into a Binary or String type (types.go)

The decode function ❶ accepts an `io.Reader` and returns a `Payload` interface and an error interface. If decode cannot decode the bytes read from the reader into a `Binary` or `String` type, it will return an error along with a nil `Payload`.

You must first read a byte from the reader ❷ to determine the type and create a payload variable ❸ to hold the decoded type. If the type you read from the reader is an expected type constant ❹, you assign the corresponding type to the payload variable.

You now have enough information to finish decoding the binary data from the reader into the payload variable by using its `ReadFrom` method. But you have a problem here. You cannot simply pass the reader to the `ReadFrom` method. You've already read a byte from it corresponding to the type, yet the `ReadFrom` method expects the first byte it reads to be the type as well. Thankfully, the `io` package has a helpful function you can use: `MultiReader`. We cover `io.MultiReader` in more detail later in this chapter, but here you use it to concatenate the byte you've already read with the reader ❺. From the `ReadFrom` method's perspective, it will read the bytes in the sequence it expects.

Although the use of `io.MultiReader` shows you how to inject bytes back into a reader, it isn't optimal in this use case. The proper fix is to remove each type's need to read the first byte in its `ReadFrom` method. Then, the `ReadFrom` method would read only the 4-byte size and the payload, eliminating the need to inject the type byte back into the reader before passing it on to `ReadFrom`. As an exercise, I recommend you refactor the code to eliminate the need for `io.MultiReader`.

Let's see the decode function in action in the form of a test. Listing 4-10 illustrates how you can send your two distinct types over a network connection and properly decode them back into their original type on the receiver's end.

```
package main

import (
    "bytes"
    "encoding/binary"
    "net"
    "reflect"
    "testing"
)

func TestPayloads(t *testing.T) {
    b1 := ❶Binary("Clear is better than clever.")
    b2 := Binary("Don't panic.")
    s1 := ❷String("Errors are values.")
    payloads := ❸[]Payload{&b1, &s1, &b2}

    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    go func() {
        conn, err := listener.Accept()
        if err != nil {
            t.Error(err)
            return
        }
        defer conn.Close()

        for _, p := range payloads {
            _, err = ❹p.WriteTo(conn)
            if err != nil {
                t.Error(err)
                break
            }
        }
    }()
}

--snip--
```

Listing 4-10: Creating the `TestPayloads` test (`types_test.go`)

Your test should first create at least one of each type. You create two Binary types ❶ and one String type ❷. Next, you create a slice of Payload interfaces and add pointers to the Binary and String types you created ❸. You then create a listener that will accept a connection and write each type in the payloads slice to it ❹.

This is a good start. Let's finish up the client side of the test in Listing 4-11.

--snip--

```
conn, err := ❶net.Dial("tcp", listener.Addr().String())
if err != nil {
    t.Fatal(err)
}
defer conn.Close()

for i := 0; i < len(payloads); i++ {
    actual, err := ❷decode(conn)
    if err != nil {
        t.Fatal(err)
    }

    ❸ if expected := payloads[i]; !reflect.DeepEqual(expected, actual) {
        t.Errorf("value mismatch: %v != %v", expected, actual)
        continue
    }

    ❹ t.Logf("[%T] %[1]q", actual)
}
}
```

Listing 4-11: Completing the TestPayloads test (types_test.go)

You know how many types to expect in the payloads slice, so you initiate a connection to the listener ❶ and attempt to decode each one ❷. Finally, your test compares the type you decoded with the type the server sent ❸. If there's any discrepancy with the variable type or its contents, the test fails. You can run the test with the -v flag to see the type and its value ❹.

Let's make sure the Binary type enforces the maximum payload size in Listing 4-12.

--snip--

```
func TestMaxPayloadSize(t *testing.T) {
    buf := new(bytes.Buffer)
    err := buf.WriteByte(BinaryType)
    if err != nil {
        t.Fatal(err)
    }

    err = binary.Write(buf, binary.BigEndian, ❶uint32(1<<30)) // 1 GB
    if err != nil {
```

```

        t.Fatal(err)
    }

    var b Binary
    _, err = b.ReadFrom(buf)
    ❷ if err != ErrMaxPayloadSize {
        t.Fatalf("expected ErrMaxPayloadSize; actual: %v", err)
    }
}

```

Listing 4-12: Testing the maximum payload size (types_test.go)

This test starts with the creation of a `bytes.Buffer` containing the `BinaryType` byte and a 4-byte, unsigned integer indicating the payload is 1GB ❶. When this buffer is passed to the `Binary` type's `ReadFrom` method, you receive the `ErrMaxPayloadSize` error in return ❷. The test cases in Listings 4-10 and 4-11 should cover the use case of a payload that is less than the maximum size, but I encourage you to modify this test to make sure that's the case.

Handling Errors While Reading and Writing Data

Unlike writing to file objects, writing to network connections can be unreliable, especially if your network connection is spotty. Files don't often return errors while you're writing to them, but the receiver on the other end of a network connection may abruptly disconnect before you write your entire payload.

Not all errors returned when reading from or writing to a network connection are permanent. The connection can recover from some errors. For example, writing data to a network connection where adverse network conditions delay the receiver's ACK packets, and where your connection times out while waiting to receive them, can result in a temporary error. This can occur if someone temporarily unplugs a network cable between you and the receiver. In that case, the network connection is still active, and you can either attempt to recover from the error or gracefully terminate your end of the connection.

Listing 4-13 illustrates how to check for temporary errors while writing data to a network connection.

```

var (
    err error
    n int
    i = 7 // maximum number of retries
)

❶ for ; i > 0; i-- {
    n, err = ❷conn.Write(❸[]byte("hello world"))
    if err != nil {
        if nErr, ok := ❹err.(net.Error); ok && ❺nErr.Temporary() {
            log.Println("temporary error:", nErr)
        }
    }
}

```

```

        time.Sleep(10 * time.Second)
        continue
    }
    ❹ return err
}
break
}

if i == 0 {
    return errors.New("temporary write failure threshold exceeded")
}

log.Printf("wrote %d bytes to %s\n", n, conn.RemoteAddr())

```

Listing 4-13: Sending the string "hello world" over the connection

Since you might receive a transient error when writing to a network connection, you might need to retry a write operation. One way to account for this is to encapsulate the code in a for loop ❶. This makes it easy to retry the write operation, if necessary.

To write to the connection, you pass a byte slice ❷ to the connection's Write method ❸ as you would to any other `io.Writer`. This returns the number of bytes written and an error interface. If the error interface is not `nil`, you check whether the error implements the `net.Error` interface by using a type assertion ❹ and check whether the error is temporary ❺. If the `net.Error`'s `Temporary` method returns `true`, the code makes another write attempt by iterating around the for loop. If the error is permanent, the code returns the error ❻. A successful write breaks out of the loop.

Creating Robust Network Applications by Using the io Package

In addition to interfaces common in Go code, such as `io.Reader` and `io.Writer`, the `io` package provides several useful functions and utilities that make the creation of robust network applications easy. In this section, you'll learn how to use the `io.Copy`, `io.MultiWriter`, and `io.TeeReader` functions to proxy data between connections, log network traffic, and ping hosts when firewalls attempt to keep you from doing so.

Proxying Data Between Connections

One of the most useful functions from the `io` package, the `io.Copy` function reads data from an `io.Reader` and writes it to an `io.Writer`. This is useful for creating a *proxy*, which, in this context, is an intermediary that transfers data between two nodes. Since `net.Conn` includes both `io.Reader` and `io.Writer` interfaces, and `io.Copy` writes whatever it reads from an `io.Reader` to an `io.Writer`, you can easily create a proxy between network connections, such

as the one you define in the `proxyConn` function in Listing 4-14. This function copies any data sent from the source node to the destination node, and vice versa.

```
package main

import (
    "io"
    "net"
)

func proxyConn(source, destination string) error {
    connSource, err := ❶net.Dial("tcp", source)
    if err != nil {
        return err
    }
    defer connSource.Close()

    connDestination, err := ❷net.Dial("tcp", destination)
    if err != nil {
        return err
    }
    defer connDestination.Close()

    // connDestination replies to connSource
    ❸ go func() { _, _ = io.Copy(connSource, connDestination) }()

    // connSource messages to connDestination
    ❹ _, err = io.Copy(connDestination, connSource)

    return err
}
```

Listing 4-14: Proxying data between two network connections (proxy_conn.go)

The `io.Copy` function does all the heavy input/output (I/O) lifting for you. It takes an `io.Writer` as its first argument and an `io.Reader` as its second argument. It then writes, to the writer, everything it reads from the reader until the reader returns an `io.EOF`, or, alternately, either the reader or writer returns an error. The `io.Copy` function returns an error only if a non-`io.EOF` error occurred during the copy, because `io.EOF` means it has read all the data from the reader.

You start by creating a connection to the source node ❶ and a connection to the destination node ❷. Next, you run `io.Copy` in a goroutine, reading from `connDestination` and writing to `connSource` ❸ to handle any replies. You don't need to worry about leaking this goroutine, since `io.Copy` will return when either connection is closed. Then, you make another call to `io.Copy`, reading from `connSource` and writing to `connDestination` ❹. Once this call returns and the function returns, each connection's `Close` method runs,

which causes `io.Copy` to return, terminating its goroutine ❸. As a result, the data is proxied between network connections as if they had a direct connection to one another.

NOTE

Since Go version 1.11, if you use `io.Copy` or `io.CopyN` when the source and destination are both `*net.TCPConn` objects, the data never enters the user space on Linux, thereby causing the data transfer to occur more efficiently. Think of it as the Linux kernel reading from one socket and writing to the other without the data needing to interact directly with your Go code. `io.CopyN` functions like `io.Copy` except it copies up to `n` bytes. We'll use `io.CopyN` in the next chapter.

Listing 4-15 illustrates how to use a slight variation of the `proxyConn` function. Whereas Listing 4-14's `proxyConn` function established network connections and proxied traffic between them, Listing 4-15's `proxy` function proxies data between an `io.Reader` and an `io.Writer`, making it applicable to more than just network connections and much easier to test.

```
package main

import (
    "io"
    "net"
    "sync"
    "testing"
)

❶ func proxy(from io.Reader, to io.Writer) error {
    fromWriter, fromIsWriter := from.(io.Writer)
    toReader, toIsReader := to.(io.Reader)

    if toIsReader && fromIsWriter {
        // Send replies since "from" and "to" implement the
        // necessary interfaces.
        go func() { _, _ = io.Copy(fromWriter, toReader) }()
    }

    _, err := io.Copy(to, from)

    return err
}
```

Listing 4-15: Proxy data between a reader and writer (`proxy_test.go`)

This `proxy` function ❶ is a bit more useful in that it accepts the ubiquitous `io.Reader` and `io.Writer` interfaces instead of `net.Conn`. Because of this change, you could proxy data from a network connection to `os.Stdout`, `*bytes.Buffer`, `*os.File`, or any number of objects that implement the `io.Writer` interface. Likewise, you could read bytes from any object that implements the `io.Reader` interface and send them to the writer. This implementation of `proxy` supports replies if the *from* reader implements the `io.Writer` interface and the *to* writer implements the `io.Reader` interface.

Listing 4-16 creates a test to make sure the `proxy` functions as you expect.

--snip--

```
func TestProxy(t *testing.T) {
    var wg sync.WaitGroup

    // server listens for a "ping" message and responds with a
    // "pong" message. All other messages are echoed back to the client.
    ❶ server, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    wg.Add(1)

    go func() {
        defer wg.Done()

        for {
            conn, err := server.Accept()
            if err != nil {
                return
            }

            go func(c net.Conn) {
                defer c.Close()

                for {
                    buf := make([]byte, 1024)
                    n, err := c.Read(buf)
                    if err != nil {
                        if err != io.EOF {
                            t.Error(err)
                        }
                    }

                    return

                    switch msg := string(buf[:n]); msg {
                    case "ping":
                        _, err = c.Write([]byte("pong"))
                    default:
                        _, err = c.Write(buf[:n])
                    }

                    if err != nil {
                        if err != io.EOF {
                            t.Error(err)
                        }
                    }

                    return
                }
            }(conn)
        }
    }()
}
```



```
}()
```

```
--snip--
```

Listing 4-16: Creating the listener (proxy_test.go)

You start by initializing a server ❶ that listens for incoming connections. It reads bytes from each connection, replies with the string "pong" when it receives the string "ping," and echoes any other message it receives.

Listing 4-17 continues the test implementation.

```
--snip--
```

```
// proxyServer proxies messages from client connections to the
// destinationServer. Replies from the destinationServer are proxied
// back to the clients.
❶ proxyServer, err := net.Listen("tcp", "127.0.0.1:")
if err != nil {
    t.Fatal(err)
}

wg.Add(1)

go func() {
    defer wg.Done()

    for {
        conn, err := ❷ proxyServer.Accept()
        if err != nil {
            return
        }

        go func(from net.Conn) {
            defer from.Close()

            to, err := ❸ net.Dial("tcp",
                server.Addr().String())
            if err != nil {
                t.Error(err)
                return
            }

            defer to.Close()

            err = ❹ proxy(from, to)
            if err != nil && err != io.EOF {
                t.Error(err)
            }
        }(conn)
    }
}()
```

```
--snip--
```

Listing 4-17: Set up the proxy between the client and server (proxy_test.go)

You then set up a proxy server ❶ that handles the message passing between the client and the destination server. The proxy server listens for incoming client connections. Once a client connection accepts ❷, the proxy establishes a connection to the destination server ❸ and starts proxying messages ❹. Since the proxy server passes two `net.Conn` objects to proxy, and `net.Conn` implements the `io.ReadWriter` interface, the server proxies replies automatically. Then `io.Copy` writes to the `Write` method of the destination `net.Conn` everything it reads from the `Read` method of the origin `net.Conn`, and vice versa for replies from the destination to the origin.

Listing 4-18 implements the client portion of the test.

--snip--

```
conn, err := net.Dial("tcp", proxyServer.Addr().String())
if err != nil {
    t.Fatal(err)
}

❶ msgs := []struct{ Message, Reply string }{
    {"ping", "pong"},
    {"pong", "pong"},
    {"echo", "echo"},
    {"ping", "pong"},
}

for i, m := range msgs {
    _, err = conn.Write([]byte(m.Message))
    if err != nil {
        t.Fatal(err)
    }

    buf := make([]byte, 1024)

    n, err := conn.Read(buf)
    if err != nil {
        t.Fatal(err)
    }

    actual := string(buf[:n])
    t.Logf("%q -> proxy -> %q", m.Message, actual)

    if actual != m.Reply {
        t.Errorf("%d: expected reply: %q; actual: %q",
            i, m.Reply, actual)
    }
}

_ = conn.Close()
_ = proxyServer.Close()
_ = server.Close()
```

```
    wg.Wait()
}
```

Listing 4-18: Proxying data from an upstream server to a downstream server (proxy_test.go)

You run the proxy through a series of tests ❶ to verify that your ping messages result in pong replies and that the destination echoes anything else you send. The output should look like the following:

```
$ go test ❶ -race -v proxy_test.go
=== RUN    TestProxy
--- PASS: TestProxy (0.00s)
    proxy_test.go:138: "ping" -> proxy -> "pong"
    proxy_test.go:138: "pong" -> proxy -> "pong"
    proxy_test.go:138: "echo" -> proxy -> "echo"
    proxy_test.go:138: "ping" -> proxy -> "pong"
PASS
ok      command-line-arguments  1.018s
```

I'm in the habit of running my tests with the `-race` flag ❶ to enable the race detector. The race detector can help alert you to data races that need your attention. Although not necessary for this test, enabling it is a good habit.

Monitoring a Network Connection

The `io` package includes useful tools that allow you to do more with network data than just send and receive it using connection objects. For example, you could use `io.MultiWriter` to write a single payload to multiple network connections. You could also use `io.TeeReader` to log data read from a network connection. Listing 4-19 gives an example of using the `io.TeeReader` and `io.MultiWriter` to log all network traffic on a TCP listener.

```
package main

import (
    "io"
    "log"
    "net"
    "os"
)

// Monitor embeds a log.Logger meant for logging network traffic.
type Monitor struct {
    *log.Logger
}

// Write implements the io.Writer interface.
func (m *Monitor) ❶Write(p []byte) (int, error) {
    return len(p), m.Output(2, string(p))
}
```

```

func ExampleMonitor() {
    ❷ monitor := &Monitor{Logger: log.New(os.Stdout, "monitor: ", 0)}

    listener, err := net.Listen("tcp", "127.0.0.1:")
    if err != nil {
        monitor.Fatal(err)
    }

    done := make(chan struct{})

    go func() {
        defer close(done)

        conn, err := listener.Accept()
        if err != nil {
            return
        }
        defer conn.Close()

        b := make([]byte, 1024)
        ❸ r := io.TeeReader(conn, monitor)

        n, err := r.Read(b)
        if err != nil && err != io.EOF {
            monitor.Println(err)
            return
        }

        ❹ w := io.MultiWriter(conn, monitor)

        _, err = w.Write(b[:n]) // echo the message
        if err != nil && err != io.EOF {
            monitor.Println(err)
            return
        }
    }()
}

--snip--

```

Listing 4-19: Using `io.TeeReader` and `io.MultiWriter` to capture a network connection's input and output (monitor_test.go)

You create a new struct named `Monitor` that embeds a `log.Logger` for the purposes of logging the server's network traffic. Since the `io.TeeReader` and the `io.MultiWriter` expect an `io.Writer`, the monitor implements the `io.Writer` interface ❶.

You start by creating an instance of `Monitor` ❷ that writes to `os.Stdout`. You use the monitor in conjunction with the connection object in an `io.TeeReader` ❸. This results in an `io.Reader` that will read from the network connection and write all input to the monitor before passing along the input to the caller. Likewise, you log server output by creating an `io.MultiWriter` ❹, writing to the network connection and the monitor.

Listing 4-20 details the client portion of the example and its output.

```
--snip--

conn, err := net.Dial("tcp", listener.Addr().String())
if err != nil {
    monitor.Fatal(err)
}

_, err = ❶ conn.Write([]byte("Test\n"))
if err != nil {
    monitor.Fatal(err)
}

_ = conn.Close()
<-done

// ❷Output:
// monitor: Test
// monitor: Test
}
```

Listing 4-20: The client implementation and example output (monitor_test.go)

When you send the message `Test\n` ❶, it's logged to `os.Stdout` twice ❷: once when you read the message from the connection, and again when you echo the message back to the client. If you want to get fancy, you could decorate the log entries to differentiate between incoming and outgoing data. One way to do this would be to create an object that implements the `io.Writer` interface and embeds the monitor. When its `Write` method is called, it prepends the data with the prefix before passing the data along to the monitor's `Write` method.

Although using the `io.TeeReader` and the `io.MultiWriter` in this fashion is powerful, it isn't without a few caveats. First, both the `io.TeeReader` and the `io.MultiWriter` will block while writing to your writer. Your writer will add latency to the network connection, so be mindful not to block too long. Second, an error returned by your writer will cause the `io.TeeReader` or `io.MultiWriter` to return an error as well, halting the flow of network data. If you don't want your use of these objects to potentially interrupt network data flow, I strongly recommend you implement a reader that always returns a nil error and logs its underlying error in a manner that's actionable.

For example, you can modify `Monitor`'s `Write` method to always return a nil error:

```
func (m *Monitor) Write(p []byte) (int, error) {
    err := m.Output(2, string(p))
    if err != nil {
        log.Println(err) // use the log package's default logger
    }

    return len(p), nil
}
```

The Monitor attempts to write the byte slice to its embedded logger. Failing that, it writes the error to the log package's default logger and returns a nil error to `io.TeeReader` and `io.MultiWriter` in Listing 4-19 so as not to interrupt the flow of data.

Pinging a Host in ICMP-Filtered Environments

In “The Internet Control Message Protocol” on page 31, you learned that ICMP is a protocol that gives you feedback about local network conditions. One of its most common uses is to determine whether a host is online by issuing a ping request and receiving a pong reply from the host. Most operating systems have a built-in ping command that sends an ICMP echo request to a destination IP address. Once the host responds with an ICMP echo reply, ping prints the duration between sending the ping and receiving the pong.

Unfortunately, many internet hosts filter or block ICMP echo replies. If a host filters pongs, the ping erroneously reports that the remote system is unavailable. One technique you can use instead is to establish a TCP connection with the remote host. If you know that the host listens for incoming TCP connections on a specific port, you can use this knowledge to confirm that the host is available, because you can establish a TCP connection only if the host is up and completes the handshake process.

Listing 4-21 shows a small application that reports the time it takes to establish a TCP connection with a host on a specific port.

```
package main

import (
    "flag"
    "fmt"
    "net"
    "os"
    "time"
)

❶ var (
    count    = flag.Int("c", 3, "number of pings: <= 0 means forever")
    interval = flag.Duration("i", time.Second, "interval between pings")
    timeout  = flag.Duration("W", 5*time.Second, "time to wait for a reply")
)

func init() {
    flag.Usage = func() {
        fmt.Printf("Usage: %s [options] host:port\nOptions:\n", os.Args[0])
        flag.PrintDefaults()
    }
}

--snip--
```

Listing 4-21: The command line flags for the ping command (ping.go)

This example starts by defining a few command line options ❶ that mimic a subset of the functionality provided by the ping command on Linux.

Listing 4-22 adds the main function.

--snip--

```
func main() {
    flag.Parse()

    if flag.NArg() != 1 {
        fmt.Println("host:port is required\n\n")
        flag.Usage()
        os.Exit(1)
    }

    target := flag.Arg(0)
    fmt.Println("PING", target)

    if *count <= 0 {
        fmt.Println("CTRL+C to stop.")
    }

    msg := 0

    for (*count <= 0) || (msg < *count) {
        msg++
        fmt.Print(msg, " ")

        start := time.Now()
        ❶ c, err := net.DialTimeout("tcp", target, *timeout)
        ❷ dur := time.Since(start)

        if err != nil {
            fmt.Printf("fail in %s: %v\n", dur, err)
            if nErr, ok := err.(net.Error); !ok || ❸ !nErr.Temporary() {
                os.Exit(1)
            }
        } else {
            _ = c.Close()
            fmt.Println(dur)
        }

        time.Sleep(*interval)
    }
}
```

Listing 4-22: Reporting the time to establish a TCP socket to a given host and port (ping.go)

You attempt to establish a connection to a remote host's TCP port ❶, setting a reasonable time-out duration if the remote host doesn't respond. You keep track of the time it takes to complete the TCP handshake and consider this duration ❷ the ping interval between your host and the remote

host. If you encounter a temporary error (for example, a time-out), you'll continue trying, and you'll exit if the error is permanent ❸. This is handy if you restart a TCP service and want to monitor its progress in restarting. Initially, the code in Listing 4-22 will report time-out errors, but it will eventually start printing valid results when the service is again listening on the specific port.

It's important to understand that system admins could consider the code in Listing 4-22 abusive, especially if you specify a large ping count. That's because you aren't simply asking the remote host to send an echo reply using ICMP. Instead, you're rapidly establishing and tearing down a TCP connection with every interval. Establishing a TCP connection has more overhead than an ICMP echo request and response. I recommend that you use this method only when intermediate firewalls filter ICMP echo messages and, even then, with the permission of the system admin.

Exploring Go's TCPConn Object

For most use cases, the `net.Conn` interface will provide adequate functionality and the best cross-platform support for TCP sessions between nodes. But accessing the underlying `net.TCPConn` object allows fine-grained control over the TCP network connection should you need to do such things as modify the read and write buffers, enable keepalive messages, or change the behavior of pending data upon closing the connection. The `net.TCPConn` object is the concrete object that implements the `net.Conn` interface. Keep in mind that not all the following functionality may be available on your target operating system.

The easiest way to retrieve the `net.TCPConn` object is by using a type assertion. This works for connections where the underlying network is TCP:

```
tcpConn, ok := conn.(*net.TCPConn)
```

On the server side, you can use the `AcceptTCP` method on a `net.TCPListener`, as shown in Listing 4-23, to retrieve the `net.TCPConn` object.

```
addr, err := net.ResolveTCPAddr("tcp", "127.0.0.1:")
if err != nil {
    return err
}

listener, err := net.ListenTCP("tcp", addr)
if err != nil {
    return err
}

tcpConn, err := listener.AcceptTCP()
```

Listing 4-23: Retrieving `net.TCPConn` from the listener

On the client side, use the `net.DialTCP` function, as shown in Listing 4-24.

```
addr, err := net.ResolveTCPAddr("tcp", "www.google.com:http")
if err != nil {
    return err
}

tcpConn, err := net.DialTCP("tcp", nil, addr)
```

Listing 4-24: Using `DialTCP` to retrieve a `net.TCPConn` object

The next few sections cover useful methods on `net.TCPConn` that are unavailable on `net.Conn`. Some of these methods may not be available on your target operating system or may have hard limits imposed by the operating system. My advice is to use the following methods only when necessary. Altering these settings on the connection object from the operating system defaults may lead to network behavior that's difficult to debug. For example, shrinking the read buffer on a network connection may lead to unexpected zero window issues unexplained by checking the operating system's default read buffer value.

Controlling Keepalive Messages

A *keepalive* is a message sent over a network connection to check the connection's integrity by prompting an acknowledgment of the message from the receiver. After an operating system–specified number of unacknowledged keepalive messages, the operating system will close the connection.

The operating system configuration dictates whether a connection uses keepalives for TCP sessions by default. If you need to enable keepalives on a `net.TCPConn` object, pass `true` to its `SetKeepAlive` method:

```
err := tcpConn.SetKeepAlive(true)
```

You also have control over how often the connection sends keepalive messages using the `SetKeepAlivePeriod` method. This method accepts a `time.Duration` that dictates the keepalive message interval:

```
err := tcpConn.SetKeepAlivePeriod(time.Minute)
```

Using deadlines advanced by a heartbeat is usually the better method for detecting network problems. As mentioned earlier in this chapter, deadlines provide better cross-platform support, traverse firewalls better, and make sure your application is actively managing the network connection.

Handling Pending Data on Close

By default, if you've written data to `net.Conn` but the data has yet to be sent to or acknowledged by the receiver and you close the network connection,

your operating system will complete the delivery in the background. If you don't want this behavior, the `net.TCPConn` object's `SetLinger` method allows you to tweak it:

```
err := tcpConn.SetLinger(-1) // anything < 0 uses the default behavior
```

With the linger disabled, it is possible that the server may receive the last portion of data you send along with your FIN when you close your connection. Since your call to `conn.Close` doesn't block, you have no way of knowing whether the server received the data you just sent prior to your FIN. It's possible the data sat in the server's receive buffer and then the server crashed, taking your unacknowledged data and FIN with it. Linger on the connection to give the server time to acknowledge the data may seem tempting. But this won't solve your problem if the server crashes, as in the example. Also, some developers may argue that using linger for this purpose is a code smell. Your application should instead verify that the server received all data before tearing down its connection if this last bit of unacknowledged data is a concern.

If you wish to abruptly discard all unsent data and ignore acknowledgments of sent data upon closing the network connection, set the connection's linger to zero:

```
err := tcpConn.SetLinger(0) // immediately discard unsent data on close
```

Setting linger to zero will cause your connection to send an RST packet when your code calls your connection's `Close` method, aborting the connection and bypassing the normal teardown procedures.

If you're looking for a happy medium and your operating system supports it, you can pass a positive integer *n* to `SetLinger`. Your operating system will attempt to complete delivery of all outstanding data up to *n* seconds, after which point your operating system will discard any unsent or unacknowledged data:

```
err := tcpConn.SetLinger(10) // discard unsent data after 10 seconds
```

If you feel compelled to modify your connection's linger value, please read up on how your operating system handles lingering on network connections. When in doubt, use the default value.

Overriding Default Receive and Send Buffers

Your operating system assigns read and write buffers to each network connection you create in your code. For most cases, those values should be enough. But in the event you want greater control over the read or write buffer sizes, you can tweak their value, as demonstrated in Listing 4-25.

```
if err := tcpConn.SetReadBuffer(212992); err != nil {  
    return err  
}
```

```
if err := tcpConn.SetWriteBuffer(212992); err != nil {
    return err
}
```

Listing 4-25: Setting read and write buffer sizes on a TCP connection

The `SetReadBuffer` method accepts an integer representing the connection's read buffer size in bytes. Likewise, the `SetWriteBuffer` method accepts an integer and sets the write buffer size in bytes on the connection. Keep in mind that you can't exceed your operating system's maximum value for either buffer size.

Solving Common Go TCP Network Problems

Go doesn't hold your hand when working with TCP network connections. As such, it's possible to introduce bugs in your code that manifest as network errors. This section presents two common TCP networking issues: zero window errors and sockets stuck in the `CLOSE_WAIT` state.

Zero Window Errors

We spent a bit of time in “Receive Buffers and Window Sizes” on page 48 discussing TCP's sliding window and how the window size tells the sender how much data the receiver can accept before the next acknowledgment. A common workflow when reading from a network connection is to read some data from the connection, handle the data, read more data from the connection, handle it, and so on.

But what happens if you don't read data from a network connection quickly enough? Eventually, the sender may fill the receiver's receive buffer, resulting in a zero-window state. The receiver will not be able to receive data until the application reads data from the buffer. This most often happens when the handling of data read from a network connection blocks and the code never makes its way around to reading from the socket again, as shown in Listing 4-26.

```
buf := make([]byte, 1024)

for {
    ❶ n, err := conn.Read(buf)
    if err != nil {
        return err
    }

    ❷ handle(buf[:n]) // BLOCKS!
}
```

Listing 4-26: Handling received data blocks preventing iteration around the loop

Reading data from the network connection ❶ frees up receive buffer space. If the code blocks for an appreciable amount of time while handling the received data ❷, the receive buffer may fill up. A full receive buffer isn't

necessarily bad. Zeroing the window is a way to *throttle*, or slow, the flow of data from the sender by creating backpressure on the sender. But if it's unintended or prolonged, a zero window may indicate a bug in your code.

Sockets Stuck in the CLOSE_WAIT State

In “Gracefully Terminating TCP Sessions” on page 50, I mentioned that the server side of a TCP network connection will enter the CLOSE_WAIT state after it receives and acknowledges the FIN packet from the client. If you see TCP sockets on your server that persist in the CLOSE_WAIT state, it's likely your code is neglecting to properly call the Close method on its network connections, as in Listing 4-27.

```
for {
    conn, err := listener.Accept()
    if err != nil {
        return err
    }

    ❶ go func(c net.Conn) { // we never call c.Close() before returning!
        buf := make([]byte, 1024)

        for {
            n, err := c.Read(buf)
            if err != nil {
                ❷ return
            }

            handle(buf[:n])
        }
    }(conn)
}
```

Listing 4-27: Returning from a connection-handling goroutine without properly closing the connection

The listener handles each connection in its own goroutine ❶. However, the goroutine fails to call the connection's Close method before fully returning from the goroutine ❷. Even a temporary error will cause the goroutine to return. And because you never close the connection, this will leave the TCP socket in the CLOSE_WAIT state. If the server attempted to send anything other than a FIN packet to the client, the client would respond with an RST packet, abruptly tearing down the connection. The solution is to make sure to defer a call to the connection's Close method soon after creating the goroutine ❶.

What You've Learned

In this chapter, you first learned several methods of reading data from and writing data to a network connection, including the type-length-value encoding scheme. You built on this knowledge and learned an efficient way

to proxy data between network connections. Next, you used a few `io` tools to monitor network traffic. Then, you used your knowledge of TCP handshakes to ping remote hosts in environments where ICMP echo requests and replies are filtered. Finally, the chapter wrapped up by covering the more platform-specific, yet powerful, methods provided by the `net.TCPConn` object and a few common connection-handling bugs.

5

UNRELIABLE UDP COMMUNICATION



Although most networking applications take advantage of TCP's reliability and flow control, the less popular User Datagram Protocol (UDP) is nonetheless an important part of the TCP/IP stack. UDP is a simple protocol with minimal features. Some applications do not require TCP's feature set and session overhead. Those applications, like domain name resolution services, opt to use UDP instead.

This chapter starts by comparing UDP to TCP, focusing on scenarios where UDP may be a better choice over TCP. Then, you'll learn how to send and receive UDP packets in Go. Finally, you'll learn why it's best to limit the size of UDP packets you send across a network and how to determine an optimal packet size.

Using UDP: Simple and Unreliable

UDP is unreliable because it does not include many of the mechanisms that make TCP so trustworthy. It provides little more than a socket address (an IP address and port). In fact, the protocol is so simple that RFC 768 describes the entire thing in about three pages. Unlike TCP, UDP does not provide session support or even confirm that the destination is accessible; it simply makes a best-effort attempt to deliver the packet. Recipients do not automatically acknowledge UDP packets, so UDP has no inherent delivery confirmation. UDP does not manage congestion, control data flow, or retransmit packets. Lastly, UDP does not guarantee that the destination receives packets in the order they originate. UDP is simply a conduit between applications and the IP layer. This simplicity is what makes UDP fast and attractive for some applications.

UDP has a few strengths over TCP. Whereas TCP must establish a session with each individual node in a group before it can transmit data, UDP can send a single packet to a group of nodes without duplicating the packet, a process known as *multicasting*. UDP is also capable of broadcasting packets to all members of a subnet since it doesn't need to establish a session between each node.

UDP is ideal when missing packets aren't detrimental to overall communication because the most recently received packets can take the place of earlier, dropped packets. Weather data is a good example of this. If you're tracking a tornado in your area by streaming weather data to your phone, you aren't as concerned about dropped packets indicating the tornado's location two minutes ago if you've received packets giving you the tornado's current location.

You should consider using UDP in your application if it doesn't require all the features TCP provides. For most network applications, TCP is the right protocol choice. But UDP is an option if its speed and simplicity better fit your use case and the reliability trade-offs aren't detrimental.

UDP's packet structure consists of an 8-byte header and a payload. The header contains 2 bytes for the source port, 2 bytes for the destination port, 2 bytes for the packet length in bytes, and a 2-byte checksum. The minimum packet length is 8 bytes to account for the header and an empty payload. Figure 5-1 illustrates the organization of a UDP packet.

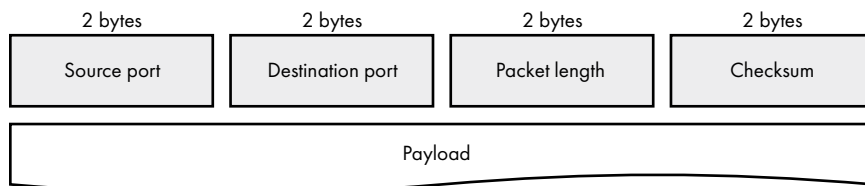


Figure 5-1: UDP packet header and payload

Although the maximum packet length is 65,535 bytes, application layer protocols often limit the packet length to avoid fragmentation, which we discuss in “Avoiding Fragmentation” on page 115.

Sending and Receiving UDP Data

When it comes to sending and receiving data, UDP is uncivilized compared to TCP. For example, let's assume your neighbor baked you a pie and wants to give it to you. Using TCP to communicate is like your neighbor shouting a greeting from her window (her socket address) to your open window (your socket address). You hear her greeting and return a greeting of your own (the TCP handshake). Your neighbor then delivers your pie. You accept it and thankfully acknowledge the pie (the data transfer). You then both exchange farewells and go about your business (the termination). By contrast, using UDP to communicate is like your neighbor abruptly throwing the pie at your window, whether it's open or not, and awaiting no confirmation that you received it.

The section “Using the `net.Conn` Interface” on page 74 introduced the `net.Conn` interface for handling stream-oriented connections, such as TCP, between a client and a server. But this interface isn't ideal for UDP connections because UDP is not a stream-oriented protocol. UDP does not maintain a session or involve a handshake process like TCP. UDP does not have the concept of acknowledgments, retransmissions, or flow control.

Instead, UDP primarily relies on the packet-oriented `net.PacketConn` interface. We'll discuss a use case for `net.Conn` with UDP later in this chapter, but `net.PacketConn` is the better choice for most UDP applications.

Using a UDP Echo Server

Sending and receiving UDP packets is a nearly identical process to sending and receiving TCP packets. But since UDP doesn't have session support, you must be able to handle an additional return value, the sender's address, when reading data from the connection object, as shown in Listing 5-1's UDP echo server implementation.

```
package echo

import (
    "context"
    "net"
)

func echoServerUDP(❶ ctx context.Context, addr string) (net.Addr, error) {
    s, err := ❷ net.ListenPacket("udp", addr)
    if err != nil {
        return nil, fmt.Errorf("binding to udp %s: %w", addr, err)
    }

    ❸ go func() {
        go func() {
            ❹ <-ctx.Done()
            _ = s.Close()
        }()

        buf := make([]byte, 1024)
```

```

    for {
        n, ❶clientAddr, err := ❸s.ReadFrom(buf) // client to server
        if err != nil {
            return
        }

        _, err = ❷s.WriteTo(buf[:n], ❸clientAddr) // server to client
        if err != nil {
            return
        }
    }
}()

return s.LocalAddr(), nil
}

```

Listing 5-1: A simple UDP echo server implementation (echo.go)

This code allows you to spin up a UDP server that will echo any UDP packets it receives to the sender. You'll make use of this code quite a bit in this chapter, so it behooves you to understand what's happening here.

The function receives a context ❶ to allow cancellation of the echo server by the caller and a string address in the familiar *host:port* format. It returns a `net.Addr` interface and an error interface. The caller uses the `net.Addr` interface to address messages to the echo server. The returned error interface is not `nil` if anything goes wrong while instantiating the echo server.

You create a UDP connection for your server with a call to `net.ListenPacket` ❷, which returns a `net.PacketConn` interface and an error interface. The `net.ListenPacket` function is analogous to the `net.Listen` function you used to create a TCP listener in Chapters 3 and 4, except `net.ListenPacket` exclusively returns a `net.PacketConn` interface.

A goroutine manages the asynchronous echoing of messages by your echo server ❸. A second goroutine blocks on the context's Done channel ❹. Once the caller cancels the context, receiving on the Done channel unblocks and the server is closed, tearing down both this goroutine and the parent goroutine ❺.

To read from the UDP connection, you pass a byte slice to the `ReadFrom` method ❸. This returns the number of bytes read, the sender's address, and an error interface. Notice there is no `Accept` method on your UDP connection as there is with the TCP-based listeners in the previous chapters. This is because UDP doesn't use a handshake process. Here, you simply create a UDP connection listening to a UDP port and read any incoming messages. Since you don't have the luxury of a proper introduction and an established session, you rely on the returned address ❶ to determine which node sent you the message.

To write a UDP packet, you pass a byte slice and a destination address ❸ to the connection's `WriteTo` method ❷. The `WriteTo` method returns the number of bytes written and an error interface. Just as with reading data, you

need to tell the `WriteTo` method where to send the packet, because you do not have an established session with a remote node. In Listing 5-1, you write the message to the original sender. But you could just as easily forward the message onto another node using your existing UDP connection object. You do not have to establish a new UDP connection object to forward on the message as you would if you were using TCP.

Receiving Data from the Echo Server

Now that you are familiar with the UDP-based echo server, let's have a look at some client code that interacts with the echo server. Listing 5-2 shows a simple interaction with the echo server.

```
package echo

import (
    "bytes"
    "context"
    "net"
    "testing"
)

func TestEchoServerUDP(t *testing.T) {
    ctx, cancel := context.WithCancel(context.Background())
    ❶ serverAddr, err := echoServerUDP(ctx, "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }
    defer cancel()

    ❷ client, err := net.ListenPacket("udp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }
    defer func() { _ = client.Close() }()

    msg := []byte("ping")
    _, err = ❸ client.WriteTo(msg, serverAddr)
    if err != nil {
        t.Fatal(err)
    }

    buf := make([]byte, 1024)
    n, ❹ addr, err := ❺ client.ReadFrom(buf)
    if err != nil {
        t.Fatal(err)
    }

    if addr.String() != serverAddr.String() {
        t.Fatalf("received reply from %q instead of %q", addr, serverAddr)
    }
}
```

```

    if !bytes.Equal(msg, buf[:n]) {
        t.Errorf("expected reply %q; actual reply %q", msg, buf[:n])
    }
}

```

Listing 5-2: Sending UDP packets to the echo server and receiving replies (echo_test.go)

You pass along a context and the address string to the `echoServer` function and receive the server's address ❶ object. You defer a call to the context's `cancel` function, which signals the server to exit and close its goroutines. In a real-world application, using a context for cancellation of long-running processes is useful to make sure you aren't leaking resources like memory or unnecessarily keeping files open.

You instantiate the client's `net.PacketConn` ❷ in the same way that you instantiated the echo server's `net.PacketConn`. The `net.ListenPacket` function creates the connection object for both the client and the server. Here, too, you need to tell the client where to send its message with each invocation of its `WriteTo` method ❸. After sending the message to the echo server, the client should immediately receive a message via its `ReadFrom` method ❹. You can examine the address ❺ returned by the `ReadFrom` method to confirm that the echo server sent the message.

It's important to note that the test in Listing 5-2 can fail under certain circumstances. Even though you're reading packets from and writing packets to a computer's local network stack, the packets are still subject to all of the conditions that make UDP unreliable over inter-node networks. For example, full send or receive buffers, or the lack of available RAM, can result in dropped packets; large UDP packets may be subject to fragmentation (discussed later in this chapter); and operating systems using multiple threads to deliver UDP packets may deliver the packets out of order.

Every UDP Connection Is a Listener

Recall from Chapter 3 that Go's `net` package distinguishes between a TCP connection object (`TCPConn`) and a TCP listener (`TCPLListener`). The TCP listener is what accepts the connection and returns an object that represents the listener's side of the connection so that the listener can then send a message to the client.

There is no UDP equivalent of the `TCPLListener` because UDP lacks sessions. This means your code has a bit more accounting to do when it receives packets. You need to verify the sender's address, because you can no longer trust that all incoming packets to a connection object are from the same sender.

The next few listings are part of a test that a single UDP connection object can receive packets from more than one sender. Listing 5-3 spins up an echo server and a client for the test.

```

package echo

import (
    "bytes"
    "context"
    "net"
    "testing"
)

func TestListenPacketUDP(t *testing.T) {
    ctx, cancel := context.WithCancel(context.Background())
    ❶ serverAddr, err := echoServerUDP(ctx, "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }
    defer cancel()

    ❷ client, err := net.ListenPacket("udp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }
    defer func() { _ = client.Close() }()
}

```

Listing 5-3: Creating an echo server and client (listen_packet_test.go)

You start by creating the echo server ❶ and client connection ❷. Listing 5-4 adds a second network connection to interact with the client.

```

--snip--

    ❶ interloper, err := net.ListenPacket("udp", "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }

    interrupt := []byte("pardon me")
    ❷ n, err := interloper.WriteTo(interrupt, client.LocalAddr())
    if err != nil {
        t.Fatal(err)
    }
    _ = interloper.Close()

    if l := len(interrupt); l != n {
        t.Fatalf("wrote %d bytes of %d", n, l)
    }
}

```

Listing 5-4: Adding an interloper and interrupting the client with a message (listen_packet_test.go)

You then create a new UDP connection ❶ meant to interlope on the client and echo server and interrupt the client ❷. This message should queue up in the client's receive buffer.

The client sends its ping message to the echo server and reconciles the replies in Listing 5-5.

--snip--

```
ping := []byte("ping")
_, err = ❶ client.WriteTo(ping, serverAddr)
if err != nil {
    t.Fatal(err)
}

buf := make([]byte, 1024)
n, addr, err := ❷ client.ReadFrom(buf)
if err != nil {
    t.Fatal(err)
}

if !bytes.Equal(❸ interrupt, buf[:n]) {
    t.Errorf("expected reply %q; actual reply %q", interrupt, buf[:n])
}

if addr.String() != interloper.LocalAddr().String() {
    t.Errorf("expected message from %q; actual sender is %q",
        interloper.LocalAddr(), addr)
}

n, addr, err = client.ReadFrom(buf)
if err != nil {
    t.Fatal(err)
}

if !bytes.Equal(❹ ping, buf[:n]) {
    t.Errorf("expected reply %q; actual reply %q", ping, buf[:n])
}

❺ if addr.String() != serverAddr.String() {
    t.Errorf("expected message from %q; actual sender is %q",
        serverAddr, addr)
}
}
```

Listing 5-5: Receiving UDP packets from multiple senders at once (listen_packet_test.go)

Meanwhile, the client writes a ping message to the echo server ❶ and promptly reads an incoming message ❷. What's unique about the UDP client connection is it first reads the interruption message from the interloping connection ❸ and then the reply from the echo server ❹. Were this a TCP connection, the client would have never received the interloper's message. As such, your code should always verify the sender of each packet it reads by evaluating the second return value ❺ from the ReadFrom method, the sender's address.

Using net.Conn in UDP

You can establish a UDP connection that implements the `net.Conn` interface so that your code behaves indistinguishably from a TCP `net.Conn`. You do so by passing `udp` as the first argument to the `net.Dial` function used in the preceding two chapters. Using `net.Conn` with your UDP-based connections can prevent interlopers from sending you messages and eliminate the need to check the sender's address on every reply you receive.

Listing 5-6 creates the UDP-based `net.Conn` and demonstrates how `net.Conn` encapsulates the implementation details of UDP to emulate a stream-oriented network connection.

```
package echo

import (
    "bytes"
    "context"
    "net"
    "testing"
    "time"
)

func TestDialUDP(t *testing.T) {
    ctx, cancel := context.WithCancel(context.Background())
    ❶ serverAddr, err := echoServerUDP(ctx, "127.0.0.1:")
    if err != nil {
        t.Fatal(err)
    }
    defer cancel()

    client, err := ❷net.Dial("udp", serverAddr.String())
    if err != nil {
        t.Fatal(err)
    }
    defer func() { _ = client.Close() }()
```

Listing 5-6: Creating an echo server and client (dial_test.go)

The client side of a connection can leverage the stream-oriented functionality of `net.Conn` over UDP, but the UDP listener must still use `net.PacketConn`. You spawn an instance of the echo server ❶ for the purpose of sending a reply to the client. You then dial the echo server over UDP by passing `udp` as the first argument to `net.Dial` ❷. Unlike TCP, the echo server receives no traffic upon calling `net.Dial` because no handshake is necessary.

Listing 5-7 interrupts the client by sending a message to it before the echo server sends its reply.

```
--snip--

interloper, err := net.ListenPacket("udp", "127.0.0.1:")
if err != nil {
    t.Fatal(err)
}
```

```

interrupt := []byte("pardon me")
❶ n, err := interloper.WriteTo(interrupt, client.LocalAddr())
if err != nil {
    t.Fatal(err)
}
_ = interloper.Close()

if l := len(interrupt); l != n {
    t.Fatalf("wrote %d bytes of %d", n, l)
}

```

Listing 5-7: Interrupting the client (dial_test.go)

Just as in Listing 5-4, you send a message to the client from an interloping connection ❶.

Listing 5-8 details the difference between a UDP connection using `net.Conn` and one using `net.PacketConn`, as in Listing 5-5.

```

--snip--

ping := []byte("ping")
_, err = ❶ client.Write(ping)
if err != nil {
    t.Fatal(err)
}

buf := make([]byte, 1024)
n, err = ❷ client.Read(buf)
if err != nil {
    t.Fatal(err)
}

if !bytes.Equal(ping, buf[:n]) {
    t.Errorf("expected reply %q; actual reply %q", ping, buf[:n])
}

err = ❸ client.SetDeadline(time.Now().Add(time.Second))
if err != nil {
    t.Fatal(err)
}

_, err = ❹ client.Read(buf)
if err == nil {
    t.Fatal("unexpected packet")
}
}

```

Listing 5-8: Using `net.Conn` to manage UDP traffic (dial_test.go)

The client sends a ping message to the echo server by using `net.Conn`'s `Write` method ❶. The `net.Conn` client will write its messages to the address specified in the `net.Dial` call. You do not need to specify a destination address for every packet you send using the client connection. Likewise, you read packets using the client's `Read` method ❷. The client reads packets only from

the sender address specified in the `net.Dial` call, as you would expect using a stream-oriented connection object. The client never reads the message sent by the interloping connection. To make sure, you set an ample deadline ❸ and attempt to read another message ❹.

For your purposes, using `net.Conn` over `net.PacketConn` may make your UDP connection code cleaner. Just be aware of the trade-offs. Using `net.Conn` with UDP does not offer the same functionality as you would expect when using `net.Conn` with TCP. For example, a UDP-based `net.Conn`'s `Write` method will not return an error if the destination failed to receive the packet. The onus is still on your application code to confirm delivery when using UDP.

Avoiding Fragmentation

Fragmentation is a Layer 3 IP process of splitting a packet into smaller pieces suitable for efficient transmission over a network. All network media have packet size limitations known as the *maximum transmission unit (MTU)*. Packets larger than the medium's maximum transmission unit require fragmentation so that each fragment is less than or equal to the medium's MTU before nodes pass them over the medium. Once the fragments reach their destination, the operating system reassembles each packet and presents the packet to your application.

But fragments can corrupt or fail to reach their destination for one reason or another. This is a significant consideration if you're using UDP because, unlike TCP, UDP does not gracefully recover from missing or corrupt data. If an operating system fails to receive even a single fragment, the sender must retransmit the entire UDP packet. As you can imagine, retransmitting large packets is woefully inefficient. Although there are numerous approaches to mitigating the effects of fragmentation, we'll attempt to avoid it altogether. We'll focus on a straightforward way to identify the MTU between your computer and a destination node, and then use those results to inform your choice of payload size to avoid fragmentation.

You can use the `ping` command to determine the MTU between your computer and a destination node. The `ping` command allows you to send an ICMP packet of a specific size with a flag set to inform nodes not to fragment it. If the packet reaches a node that needs to fragment the packet because of its size, the node will see the *do not fragment* flag and respond with an ICMP message informing you that the packet is too large.

The following example sends these pings over Ethernet, which has a minimum MTU of 46 bytes and a maximum MTU of 1,500 bytes, per its specification. If any hop between your computer and its destination has an MTU of less than 1,500 bytes, your packet will fragment. Let's confirm that with the `ping` command on Linux (Listing 5-9).

```
$ ping -M ❶do -s ❷1500 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 1500(❸1528) bytes of data.
ping: sendmsg: ❹Message too long
```

Listing 5-9: Pinging 1.1.1.1 with a payload size of 1,500 bytes on Linux

You set the `-M` flag ❶ to do, which sets the prohibit fragmentation option, and set the `-s` flag ❷ to 1500, which sets a payload of 1,500 bytes. Since you aren't accounting for the packet's header size, this should exceed the Ethernet MTU. As expected, you get a notification that the packet needs fragmentation ❸. You also see that the total packet size is 1,528 bytes ❹. The extra 28 bytes is the sum of the 8-byte ICMP header and the 20-byte IP header. Any payload you specify should account for the overall header size as well.

As you can see, you never received a reply from 1.1.1.1 in Listing 5-9 because the packet you sent was too big to traverse each hop without requiring fragmentation. Instead, the `ping` command informed you that your message was too long.

Let's try again and subtract 28 bytes from the payload (Listing 5-10).

```
$ ping -M do -s 1472 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 1472(1500) bytes of data.
1480 bytes from 1.1.1.1: icmp_seq=1 ttl=59 time=11.8 ms
```

Listing 5-10: Pinging 1.1.1.1 with a payload size of 1472 bytes on Linux

That's more like it. You confirmed that the MTU between this computer and 1.1.1.1 over the internet is set to 1,500 bytes. This is the maximum packet size you can send on the network before it will require fragmentation. Thankfully, the UDP header is also 8 bytes, so the `ping` command gives accurate results despite using ICMP. Accounting for the headers, your maximum UDP payload size is 1,472 bytes to avoid fragmentation.

The equivalent `ping` command on Windows is the following:

```
C:\>ping -f -l 1500 1.1.1.1
```

The `-f` flag instructs nodes to not fragment the packet, and the `-l` flag sets the packet size to the given integer in bytes.

On macOS, the `ping` command looks like this:

```
$ ping -D -s 1500 1.1.1.1
```

The `-D` flag sets the no fragmentation flag, and the `-s` flag specifies the payload size.

Keep in mind that the MTU from your computer may differ from the examples in this chapter because of MTU settings either on your network or between your computer and the destination you choose to ping. I recommend you experiment with the `ping` command to determine the MTU from your computer to various hosts on the internet and see if you find any differences.

What You've Learned

UDP is a minimalistic, datagram-based protocol that favors speed over reliability by eschewing many of TCP's flow control and reliability features. UDP is ideal when speed and simplicity are required and the potential for data loss is acceptable, such as with live video streaming.

Since UDP is not session based, there is no concept of a UDP listener that accepts a connection after establishing a session. Rather, you create a network connection by using `net.ListenPacket`, which returns a `net.PacketConn` interface. Your code can then read any incoming messages, or datagrams, from the `net.PacketConn` interface since every `net.PacketConn` listens for incoming messages.

Fragmentation is a serious consideration when using UDP. It's important to avoid fragmentation of your UDP packets whenever possible to help ensure delivery. The `ping` command can help you derive the appropriate maximum transmission unit between your computer and your destination network. Since ICMP packet headers used by the `ping` command are the same size as UDP headers, you can use that knowledge to easily determine the payload size threshold at which fragmentation will occur. Aside from managing fragmentation by appropriately sizing your payloads, your code must manage acknowledgments and retransmissions to ensure reliability.

6

ENSURING UDP RELIABILITY



Chapter 5 introduced basic network applications using UDP and demonstrated the flexibility of Go's `net` package and interfaces for writing portable code. This chapter picks up where the last one left off to introduce one method of ensuring reliability when communicating over UDP.

This chapter starts by introducing an application protocol built on top of UDP. We'll cover a subset of types used by this protocol and demonstrate how they are used to reliably transfer data. We'll then implement a server that allows clients to download files using the application protocol. Finally, we'll download a file from our server and verify its integrity.

Reliable File Transfers Using TFTP

As discussed in the preceding chapter, UDP is inherently unreliable. That means it's your application's job to make the UDP connection reliable.

Since we spent the last chapter covering UDP and how it's best used in situations that require a subset of TCP features, it's only appropriate that we look at an example of such an application-level protocol.

The *Trivial File Transfer Protocol (TFTP)* is an example of an application protocol that ensures reliable data transfers over UDP. It allows two nodes to transfer files over UDP by implementing a subset of the features that make TCP reliable. A TFTP server implements ordered packet delivery, acknowledgments, and retransmissions. To distill this example down to the essential bits, your server allows clients to download binary data only. It does not support uploads, American Standard Code for Information Interchange (ASCII) transfers, or some of the later additions to TFTP specified outside RFC 1350. Your server expediently serves the same file, no matter what file the client requests, in the name of simplicity.

Please keep in mind that TFTP is not appropriate for secure file transmission. Though it adds reliability to UDP connections, it does not support encryption or authentication. If your application requires communication over UDP, you may want to use WireGuard (<https://github.com/WireGuard/wireguard-go/>), an application that allows for secure communication over UDP.

The next few sections will implement a read-only TFTP server to teach you the basics of adding reliability to UDP. By *read-only*, I mean your server will allow clients to only download files, not upload them. You will start by defining the subset of constants and types your TFTP server supports. You will encapsulate as much of the type-related logic in each type's methods. You'll then implement the TFTP server portion of the code that will interact with clients and use the types we define to facilitate reliable file transfers.

TFTP Types

Your TFTP server will accept read requests from the client, send data packets, transmit error packets, and accept acknowledgments from the client. To do this, you must define a few types in your code to represent client requests, transmitted data, acknowledgments, and errors. Listing 6-1 outlines key types used to cap packet sizes, identify operations, and codify various errors.

```
package tftp

import (
    "bytes"
    "encoding/binary"
    "errors"
    "io"
    "strings"
)

const (
    DatagramSize = ❶516 // the maximum supported datagram size
    BlockSize = ❷DatagramSize - 4 // the DatagramSize minus a 4-byte header
)
```

```

❸ type OpCode uint16

const (
    OpRRQ OpCode = iota + 1
    // no WRQ support
    OpData
    OpAck
    OpErr
)

❹ type ErrCode uint16

const (
    ErrUnknown ErrCode = iota
    ErrNotFound
    ErrAccessViolation
    ErrDiskFull
    ErrIllegalOp
    ErrUnknownID
    ErrFileExists
    ErrNoUser
)

```

Listing 6-1: Types and codes used by the TFTP server (types.go)

TFTP limits datagram packets to 516 bytes or fewer to avoid fragmentation. You define two constants to enforce the datagram size limit ❶ and the maximum data block size ❷. The maximum block size is the datagram size minus a 4-byte header. The first 2 bytes of a TFTP packet's header is an operation code ❸.

Each operation code is a 2-byte, unsigned integer. Your server supports four operations: a read request (RRQ), a data operation, an acknowledgment, and an error. Since your server is read-only, you skip the write request (WRQ) definition.

As with the operation codes, you define a series of unsigned 16-bit integer error codes ❹ per the RFC. Although you don't use all error codes in your server since it allows only downloads, a client could return these error codes in lieu of an acknowledgment packet.

The following sections detail the types that implement your server's four supported operations.

Read Requests

The server receives a *read request* packet when the client wants to download a file. The server must then respond with either a data packet or an error packet, both of which you'll look at in the next few sections. Either packet serves as an acknowledgment to the client that the server received the read request. If the client does not receive a data or error packet, it may retransmit the read request until the server responds or the client gives up.

Figure 6-1 illustrates the structure of a read request packet.



Figure 6-1: Read request packet structure

The read request packet consists of a 2-byte operation code, a filename, a null byte, a mode, and a trailing null byte. An *operation code* is an integer that is unique to each of your operation types. Each type's operation code corresponds to the integer detailed in RFC 1350. For example, a read request's operation code is 1. The filename and mode are strings of varying lengths. The mode indicates to the server how it should send the file: *netascii* or *octet*. If a client requests a file using the *netascii* mode, the client must convert the file to match its own line-ending format. For our purposes, you will accept only the *octet* mode, which tells the server to send the file in a binary format, or as is.

Listing 6-2 is a continuation of Listing 6-1. Here, you define the read request and its method that allows the server to marshal the request into a slice of bytes in preparation for writing to a network connection.

--snip--

```
❶ type ReadReq struct {
    Filename string
    Mode     string
}

// Although not used by our server, a client would make use of this method.
func (q ReadReq) MarshalBinary() ([]byte, error) {
    mode := "octet"
    if q.Mode != "" {
        mode = q.Mode
    }

    // operation code + filename + 0 byte + mode + 0 byte
    cap := 2 + 2 + len(q.Filename) + 1 + len(q.Mode) + 1

    b := new(bytes.Buffer)
    b.Grow(cap)

    err := ❷binary.Write(b, binary.BigEndian, OpRRQ) // write operation code
    if err != nil {
        return nil, err
    }

    _, err = b.WriteString(q.Filename) // write filename
    if err != nil {
        return nil, err
    }

    err = ❸b.WriteByte(0) // write 0 byte
    if err != nil {
```



```

        return nil, err
    }

    _, err = b.WriteString(mode) // write mode
    if err != nil {
        return nil, err
    }

    err = ❸b.WriteByte(0) // write 0 byte
    if err != nil {
        return nil, err
    }

    return b.Bytes(), nil
}

```

Listing 6-2: Read request and its binary marshaling method (types.go continued)

The struct representing your read request ❶ needs to keep track of the filename and the mode. You insert the operation code ❷ and null bytes ❸ into the buffer while marshaling the packet to a byte slice.

Listing 6-3 continues where Listing 6-2 left off and rounds out the read request's implementation by defining a method that allows the server to unmarshal a read request from a byte slice, typically read from a network connection with a client.

--snip--

```

func (q *ReadReq) ❶UnmarshalBinary(p []byte) error {
    r := bytes.NewBuffer(p)

    var code OpCode

    err := ❷binary.Read(r, binary.BigEndian, &code) // read operation code
    if err != nil {
        return err
    }

    if code != OpRRQ {
        return errors.New("invalid RRQ")
    }

    q.Filename, err = ❸r.ReadString(0) // read filename
    if err != nil {
        return errors.New("invalid RRQ")
    }

    q.Filename = ❹strings.TrimRight(q.Filename, "\x00") // remove the 0-byte
    if len(q.Filename) == 0 {
        return errors.New("invalid RRQ")
    }

    q.Mode, err = r.ReadString(0) // read mode
    if err != nil {

```

```

        return errors.New("invalid RRQ")
    }

    q.Mode = strings.TrimRight(q.Mode, "\x00") // remove the 0-byte
    if len(q.Mode) == 0 {
        return errors.New("invalid RRQ")
    }

    actual := strings.ToLower(q.Mode) // enforce octet mode
    if actual != "octet" {
        return errors.New("only binary transfers supported")
    }

    return nil
}

```

Listing 6-3: Read request type implementation (types.go continued)

Your TFTP server's read request, data, acknowledgment, and error packets all implement the `encoding.BinaryMarshaler` and `encoding.BinaryUnmarshaler` interfaces. These methods allow your types to marshal themselves to a binary format suitable for transmission over the network and from network bytes back into the original types. For example, the read request type can marshal itself into a byte slice that matches the read request format showed in Figure 6-1 by using its `MarshalBinary` method from Listing 6-2. Likewise, it can constitute itself from a byte slice read from the network using its `UnmarshalBinary` method ❶. Although your server does not send a read request and make use of its `MarshalBinary` method, I encourage you to write a TFTP client that will marshal a read request to its binary form as you progress through this chapter. I leave it as an exercise for you to implement.

The `UnmarshalBinary` method returns `nil` only if the given byte slice matches the read request format. If you are unsure of whether a given byte slice is a read request, you can pass the byte slice to this method and make that determination based on the return value. You will see this in action when you look at the server code.

The `UnmarshalBinary` method reads in the first 2 bytes ❷ and confirms the operation code is that of a read request. It then reads all bytes up to the first null byte ❸ and strips the null byte delimiter ❹. The resulting string of bytes represents the filename. Similarly, you read in the mode, returning `nil` if everything is as expected. The server can then use the populated `ReadReq` to retrieve the requested file for the client.

Data Packets

Clients receive *data packets* in response to their read requests, provided the server was able to retrieve the requested file. The server sends the file in a series of data packets, each of which has an assigned block number, starting at 1 and incrementing with every subsequent data packet. The block number allows the client to properly order the received data and account for duplicates.

All data packets have a payload of 512 bytes except for the last packet. The client continues to read data packets until it receives a data packet whose payload is less than 512 bytes, indicating the end of the transmission. At any point, the client can return an error packet in place of an acknowledgment, and the server can return an error packet instead of a data packet. An error packet immediately terminates the transfer.

Figure 6-2 shows the format of a data packet.

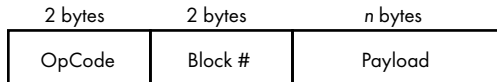


Figure 6-2: Data packet structure

Like the read request packet, the data packet's first 2 bytes contain its operation code. The next 2 bytes represent the block number. The remaining bytes, up to 512, are the payload.

The server requires an acknowledgment from the client after each data packet. If the server does not receive a timely acknowledgment or an error from the client, the server will retry the transmission until it receives a reply or exhausts its number of retries. Figure 6-3 illustrates the initial communication between a client downloading a file from a TFTP server.

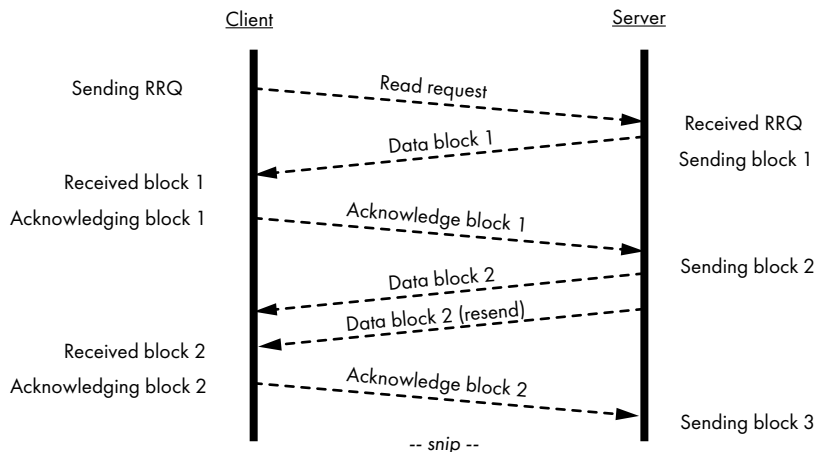


Figure 6-3: Downloading a file by using the Trivial File Transfer Protocol

Once the client has sent the initial read request packet, the server responds with the first block of data. Next, the client acknowledges receipt of block 1. The server receives the acknowledgment and replies with the second block of data. But in this contrived example, the server does not receive a timely reply from the client, so it resends block 2. The client receives block 2 and sends its acknowledgment. This back-and-forth continues until the server sends the last block with a payload of fewer than 512 bytes.

Listing 6-4 details the data type that is used for the actual data transfer.

```
--snip--

❶ type Data struct {
    Block    uint16
    Payload io.Reader
}

❷ func (d *Data) MarshalBinary() ([]byte, error) {
    b := new(bytes.Buffer)
    b.Grow(DatagramSize)

    d.Block++ // block numbers increment from 1

    err := binary.Write(b, binary.BigEndian, OpData) // write operation code
    if err != nil {
        return nil, err
    }

    err = binary.Write(b, binary.BigEndian, d.Block) // write block number
    if err != nil {
        return nil, err
    }

    // write up to BlockSize worth of bytes
    _, err = ❸io.CopyN(b, d.Payload, BlockSize)
    if err != nil && err != io.EOF {
        return nil, err
    }

    return b.Bytes(), nil
}
```

Listing 6-4: Data type and its binary marshaling method (types.go continued)

The Data struct ❶ keeps track of the current block number and the data source. In this case, your payload is an `io.Reader` instead of a byte slice, the reasoning being that an `io.Reader` allows greater flexibility about where you retrieve the payload. You could just as easily use an `*os.File` object to read a file from the filesystem as you could use a `net.Conn` to read the data from another network connection. The `io.Reader` interface gives you options that a simple byte slice doesn't. You're relying on the reader to keep track of the bytes left to read, eliminating a lot of code you'd otherwise have to write.

Every call to `MarshalBinary` ❷ will return 516 bytes per call at most by relying on the `io.CopyN` function ❸ and the `BlockSize` constant. Since you want `MarshalBinary` to modify the state, you need to use a pointer receiver. The intention is that the server can keep calling this method to get sequential blocks of data, each with an increasing block number, from the `io.Reader` until it exhausts the reader. Just like the client, the server needs to monitor

the packet size returned by this method. When the packet size is less than 516 bytes, the server knows it received the last packet and should stop calling `MarshalBinary`. You'll see this in action in the server code later in this chapter.

You may have recognized the potential for an integer overflow of the 16-bit, unsigned block number. If you send a payload larger than about 33.5MB ($65,535 \times 512$ bytes), the block number will overflow back to 0. Your server will happily continue sending data packets, but the client may not be as graceful handling the overflow. You should consider mitigating overflow risks by limiting the file size the TFTP server will support so as not to trigger the overflow, recognizing that an overflow can occur and determining whether it is acceptable to the client, or using a different protocol altogether.

Listing 6-5 finishes up the data type implementation with its binary unmarshaling method. This method follows the code in Listing 6-4.

--snip--

```
func (d *Data) UnmarshalBinary(p []byte) error {
    ❶ if l := len(p); l < 4 || l > DatagramSize {
        return errors.New("invalid DATA")
    }

    var opcode

    err := ❷ binary.Read(bytes.NewReader(p[:2]), binary.BigEndian, &opcode)
    if err != nil || opcode != OpData {
        return errors.New("invalid DATA")
    }

    err = ❸ binary.Read(bytes.NewReader(p[2:4]), binary.BigEndian, &d.Block)
    if err != nil {
        return errors.New("invalid DATA")
    }

    d.Payload = ❹ bytes.NewBuffer(p[4:])

    return nil
}
```

Listing 6-5: Data type implementation (types.go continued)

To unmarshal data, you perform an initial sanity check ❶ to determine whether the packet size is within the expected bounds, making it worth reading the remaining bytes. You then read the operation code ❷ and check it, then the block number ❸. Finally, you stuff the remaining bytes into a new buffer ❹ and assign it to the `Payload` field.

The client uses the block number to send a corresponding acknowledgment to the server and to properly order this block of data among the other received blocks of data.

Acknowledgments

Acknowledgment packets are only 4 bytes long, as shown in Figure 6-4.

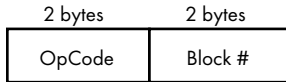


Figure 6-4: Acknowledgment packet structure

As in the other types, the first 2 bytes represent the operation code. The final 2 bytes contain the number of the acknowledged block.

Listing 6-6 shows the entire implementation of the acknowledgment type, which follows Listing 6-5's code.

--snip--

❶ type Ack uint16

```
func (a Ack) MarshalBinary() ([]byte, error) {
    cap := 2 + 2 // operation code + block number

    b := new(bytes.Buffer)
    b.Grow(cap)

    err := binary.Write(b, binary.BigEndian, OpAck) // write operation code
    if err != nil {
        return nil, err
    }

    err = binary.Write(b, binary.BigEndian, a) // write block number
    if err != nil {
        return nil, err
    }

    return b.Bytes(), nil
}

func (a *Ack) UnmarshalBinary(p []byte) error {
    var code OpCode

    r := bytes.NewReader(p)

    err := binary.Read(r, binary.BigEndian, &code) // read operation code
    if err != nil {
        return err
    }

    if code != OpAck {
        return errors.New("invalid ACK")
    }
}
```

```

    return binary.Read(r, binary.BigEndian, a) // read block number
}

```

Listing 6-6: Acknowledgment type implementation (types.go continued)

You represent an acknowledgment packet by using a 16-bit, unsigned integer ❶. This integer is set to the acknowledged block number. The `MarshalBinary` and `UnmarshalBinary` methods should look familiar by this point. They handle marshaling the operation code and block number to a byte slice and populating an `Ack` object from bytes read from the network, respectively.

Handling Errors

In TFTP, clients and servers convey errors by using an *error packet*, illustrated in Figure 6-5.

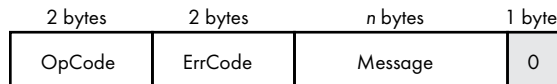


Figure 6-5: Error packet structure

Error packets consist of a 2-byte operation code, a 2-byte error code, an error message of variable length, and a terminating null byte.

Listing 6-7 details the error type and its binary marshal method, a continuation of Listing 6-6.

--snip--

```

❶ type Err struct {
    Error  ErrCode
    Message string
}

func (e Err) MarshalBinary() ([]byte, error) {
    // operation code + error code + message + 0 byte
    cap := 2 + 2 + len(e.Message) + 1

    b := new(bytes.Buffer)
    b.Grow(cap)

    err := binary.Write(b, binary.BigEndian, OpErr) // write operation code
    if err != nil {
        return nil, err
    }

    err = binary.Write(b, binary.BigEndian, e.Error) // write error code
    if err != nil {
        return nil, err
    }

    _, err = b.WriteString(e.Message) // write message
    if err != nil {

```

```

        return nil, err
    }

    err = b.WriteByte(0) // write 0 byte
    if err != nil {
        return nil, err
    }

    return b.Bytes(), nil
}

```

Listing 6-7: Error type used for conveying errors between the client and server (types.go continued)

Like the read request, the error type ❶ contains the minimum data required to craft an error packet: an error code and an error message. The `MarshalBinary` method populates a bytes buffer following the byte sequence detailed in Figure 6-5.

Listing 6-8 completes the error type implementation with its binary unmarshaler method. This code is appended to the code in Listing 6-7.

--snip--

```

func (e *Err) UnmarshalBinary(p []byte) error {
    r := bytes.NewBuffer(p)

    var code OpCode

    err := ❶binary.Read(r, binary.BigEndian, &code) // read operation code
    if err != nil {
        return err
    }

    if code != OpErr {
        return errors.New("invalid ERROR")
    }

    err = ❷binary.Read(r, binary.BigEndian, &e.Error) // read error message
    if err != nil {
        return err
    }

    e.Message, err = ❸r.ReadString(0)
    e.Message = ❹strings.TrimRight(e.Message, "\x00") // remove the 0-byte

    return err
}

```

Listing 6-8: Error type's binary unmarshaler implementation (types.go continued)

The `UnmarshalBinary` method is quite simple in that it reads and verifies the operation code ❶, consumes the error code ❷ and error message ❸, and strips the trailing null byte ❹.

The TFTP Server

Now you'll write the server code, which will use the types you defined to interact with TFTP clients.

Writing the Server Code

Listing 6-9 describes your server type and the methods that allow it to serve incoming requests. The fact that your packet types implement the encoding `.BinaryMarshaler` and encoding `.BinaryUnmarshaler` interfaces means that your server code can act as a conduit between the network interface and these types, leading to simpler code. All your server must concern itself with is transferring byte slices between your types and the network connection. The logic in the type interfaces takes care of the rest.

```
package tftp

import (
    "bytes"
    "errors"
    "fmt"
    "log"
    "net"
    "time"
)

type Server struct {
    ❶ Payload []byte // the payload served for all read requests
    ❷ Retries uint8 // the number of times to retry a failed transmission
    ❸ Timeout time.Duration // the duration to wait for an acknowledgment
}

func (s Server) ListenAndServe(addr string) error {
    conn, err := net.ListenPacket("udp", addr)
    if err != nil {
        return err
    }
    defer func() { _ = conn.Close() }()

    log.Printf("Listening on %s ...\n", conn.LocalAddr())

    return s.Serve(conn)
}

func (s *Server) ❹ Serve(conn net.PacketConn) error {
    if conn == nil {
        return errors.New("nil connection")
    }

    if s.Payload == nil {
        return errors.New("payload is required")
    }
}
```

```

    if s.Retries == 0 {
        s.Retries = 10
    }

    if s.Timeout == 0 {
        s.Timeout = 6 * time.Second
    }

    var rrq ReadReq

    for {
        buf := make([]byte, DatagramSize)

        _, addr, err := conn.ReadFrom(buf)
        if err != nil {
            return err
        }

        err = ❸ rrq.UnmarshalBinary(buf)
        if err != nil {
            log.Printf("[%s] bad request: %v", addr, err)
            continue
        }

        ❹ go s.handle(addr.String(), rrq)
    }
}

```

Listing 6-9: Server type implementation (server.go)

Our server maintains a payload ❶ that it returns for every read request, a record of the number of times to attempt packet delivery ❷, and a time-out duration between each attempt ❸. The server's `Serve` method accepts a `net.PacketConn` and uses it to read incoming requests ❹. Closing the network connection will cause the method to return.

The server reads up to 516 bytes from its connection and attempts to unmarshal the bytes to a `ReadReq` object ❺. Since your server is read-only, it's interested only in servicing read requests. If the data read from the connection is a read request, the server passes it along to a handler method in a goroutine ❻. We'll define that next.

Handling Read Requests

The handler (Listing 6-10) accepts read requests from the client and replies with the server's payload. It uses the features you built into your TFTP server's type system to improve the reliability of the data transfer over UDP. The handler sends one data packet and waits for an acknowledgment from the client before sending another data packet. It also attempts to retransmit the current data packet when it fails to receive a timely reply from the client.

--snip--

```
❶ func (s Server) handle(clientAddr string, rrq ReadReq) {
    log.Printf("[%s] requested file: %s", clientAddr, rrq.Filename)

    conn, err := ❷net.Dial("udp", clientAddr)
    if err != nil {
        log.Printf("[%s] dial: %v", clientAddr, err)
        return
    }
    defer func() { _ = conn.Close() }()

    var (
        ackPkt  Ack
        errPkt  Err
        dataPkt = ❸Data{Payload: bytes.NewReader(s.Payload)}
        buf     = make([]byte, DatagramSize)
    )

NEXTPACKET:
    ❹ for n := DatagramSize; n == DatagramSize; {
        data, err := dataPkt.MarshalBinary()
        if err != nil {
            log.Printf("[%s] preparing data packet: %v", clientAddr, err)
            return
        }

RETRY:
        ❺ for i := s.Retries; i > 0; i-- {
            ❻ n, err = conn.Write(data) // send the data packet
            if err != nil {
                log.Printf("[%s] write: %v", clientAddr, err)
                return
            }

            // wait for the client's ACK packet
            _ = conn.SetReadDeadline(time.Now().Add(s.Timeout))

            _, err = conn.Read(buf)
            if err != nil {
                if nErr, ok := err.(net.Error); ok && nErr.Timeout() {
                    continue RETRY
                }

                log.Printf("[%s] waiting for ACK: %v", clientAddr, err)
                return
            }

            switch {
            case ackPkt.UnmarshalBinary(buf) == nil:
                ❼ if uint16(ackPkt) == dataPkt.Block {
                    // received ACK; send next data packet
                    continue NEXTPACKET
                }
            }
        }
    }
}
```

```

    }
    case errPkt.UnmarshalBinary(buf) == nil:
        log.Printf("[%s] received error: %v",
            clientAddr, errPkt.Message)
        return
    default:
        log.Printf("[%s] bad packet", clientAddr)
    }
}

log.Printf("[%s] exhausted retries", clientAddr)
return
}

log.Printf("[%s] sent %d blocks", clientAddr, dataPkt.Block)
}

```

Listing 6-10: Handling read requests (server.go continued)

This handler is a method ❶ on your `Server` type that accepts a client address and a read request. It's defined as a method because you need access to the `Server`'s fields. You then initiate a connection with the client by using `net.Dial` ❷. The resulting UDP connection object created with `net.Dial`, if you remember, will read only packets from the client, freeing you from having to check the sender address on every `Read` call. You prepare a data object ❸ by using the server's payload, then enter a `for` loop to send each data packet ❹. This `for` loop will continue looping as long as the data packet size is equal to 516 bytes.

After marshaling the data object to a byte slice, you enter the `for` loop ❺ meant to resend the data packet until you either exhaust the number of retries or successfully deliver the data packet. Writing the data packet to the network connection ❻ updates the `n` loop variable with the number of bytes sent. If this value is 516 bytes, you iterate again when control passes back to the `for` loop ❹ labeled `NEXTPACKET`. If this value is less than 516 bytes, you break out of the loop.

Before you determine whether the transfer is complete, you must first verify that the client successfully received the last data packet. You read bytes from the client and attempt to unmarshal them into an `Ack` object or `Err` object. If you successfully unmarshal them into an `Err` object, you know the client returned an error, so you should log that fact and return early. An early return from this handler means the transmission terminated short of sending the entire payload. For our purposes, this is unrecoverable. The client would need to re-request the file to initiate another transfer.

If you successfully unmarshal the bytes into an `Ack` object, you can then check the object's `Block` value to determine whether it matches the block number of the current data packet ❼. If so, you iterate around the `for` loop ❹ and send the next packet. If not, you iterate around the inner `for` loop ❺ and resend the current data packet.

Starting the Server

To start your TFTP server, you need to give the server two things: a file (its payload) and an address on which to listen for incoming requests (Listing 6-11).

```
package main

import (
    "flag"
    "io/ioutil"
    "log"

    "github.com/awoodbeck/gnp/ch06/tftp"
)

var (
    address = flag.String("a", "127.0.0.1:69", "listen address")
    payload = flag.String("p", "payload.svg", "file to serve to clients")
)

func main() {
    flag.Parse()

    p, err := ❶ ioutil.ReadFile(*payload)
    if err != nil {
        log.Fatal(err)
    }

    s := ❷ tftp.Server{Payload: p}
    ❸ log.Fatal(s.ListenAndServe(*address))
}
```

Listing 6-11: Command line TFTP server implementation (tftp.go)

Once you've read the file ❶ that your TFTP server will serve into a byte slice, you instantiate the server and assign the byte slice to the server's Payload field ❷. The last step is calling its ListenAndServe method to establish the UDP connection on which it will listen for requests. The ListenAndServe method ❸ calls the server's Serve method for you, which listens on the network connection for incoming requests. The server will continue to run until you terminate it with a CTRL-C on the command line.

Downloading Files over UDP

Now let's try to download a file from the server you just wrote. First, you need to make sure you have a TFTP client installed. Windows has a native TFTP client that you can install through the Programs and Features section of the Control Panel by clicking the Turn Windows features on or off link. Select the **TFTP Client** checkbox and click the **OK** button to install it. Most

Linux distributions have a TFTP client available for installation through the distribution's package manager, and macOS has a TFTP client installed by default.

This example uses Windows 10. Start by running the TFTP server by running the code in Listing 6-11 in a terminal:

```
Microsoft Windows [Version 10.0.18362.449]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\User\gnp\ch06\tftp\tftp>go run tftp.go
2006/01/02 15:04:05 Listening on 127.0.0.1:69 ...
```

The server should bind to UDP port 69 on 127.0.0.1 by default. Port 69 is a privileged port, and you may need root permissions on Linux. You may need to first build the binary by using `go build tftp.go` and then run the resulting binary by using the `sudo` command to bind to port 69: `sudo ./tftp`. The TFTP server should log a message to standard output that indicates it's listening.

From a separate terminal, execute the TFTP client, making sure to pass the `-i` argument to tell the server you wish to initiate a binary (octet) transfer. Remember, your TFTP server doesn't care what the source filename is because it returns the same payload regardless of the requested filename. You'll use *test.svg* in this example:

```
Microsoft Windows [Version 10.0.18362.449]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\User>tftp -i 127.0.0.1 GET test.svg
Transfer successful: 75352 bytes in 1 second(s), 75352 bytes/s
```

Almost immediately upon pressing `ENTER`, the client should report the transfer was successful. The TFTP server's terminal should show its progress as well:

```
Microsoft Windows [Version 10.0.18362.449]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\User\gnp\ch06\tftp\tftp>go run tftp.go
2006/01/02 15:04:05 Listening on 127.0.0.1:69 ...
2006/01/02 15:04:05 [127.0.0.1:57944] requested file: test.svg
2006/01/02 15:04:05 [127.0.0.1:57944] sent 148 blocks
```

You can confirm that the downloaded file is the same as the payload provided to the TFTP server by comparing *test.svg*'s checksum with the checksum of the server's *payload.svg*. A *checksum* is a calculated value used to verify the integrity of a file. If two files are identical, they will have equivalent checksums. Linux and macOS both have various command line utilities for generating checksums, but you'll use a pure Go implementation, as shown in Listing 6-12.

```

package main

import (
    "crypto/sha512"
    "flag"
    "fmt"
    "io/ioutil"
    "os"
)

func init() {
    flag.Usage = func() {
        fmt.Printf("Usage: %s file...\n", os.Args[0])
        flag.PrintDefaults()
    }
}

func main() {
    flag.Parse()
    for _, file := range ❶flag.Args() {
        fmt.Printf("%s %s\n", checksum(file), file)
    }
}

func checksum(file string) string {
    b, err := ❷ioutil.ReadFile(file)
    if err != nil {
        return err.Error()
    }

    return fmt.Sprintf("%x", ❸sha512.Sum512_256(b))
}

```

Listing 6-12: Generating SHA512/256 checksums for given command line arguments (sha512-256sum.go)

This bit of code will accept one or more file paths as command line arguments ❶ and generate SHA512/256 checksums ❸ from their contents ❷.

A SHA512/256 checksum is a SHA512 checksum truncated to 256 bits. Calculating SHA512 on a 64-bit machine is faster than calculating a SHA256 checksum, because the SHA512 computation uses 64-bit words, whereas SHA256 uses 32-bit words. By truncating SHA512 to 256 bits, you eliminate a length extension hashing attack that SHA512 is vulnerable to by itself. SHA512/256 isn't necessary here since you're not using the checksum beyond verifying the integrity of a file, but you should be familiar with it, and it should be on your short list of hashing algorithms.

You can use the code from Listing 6-12 in Listing 6-13 to verify that the file you downloaded (*test.svg*) is identical to the file the server sent (*payload.svg*).

You'll continue to use Windows as your target platform, but the code will work on Linux and macOS without changes:

Microsoft Windows [Version 10.0.18362.449]

(c) 2019 Microsoft Corporation. All rights reserved.

```
C:\Users\User\dev\gnp\ch06>go build sha512-256sum\sha512-256sum.go
```

```
C:\Users\User\dev\gnp\ch06>sha512-256sum \Users\User\test.svg
```

```
\Users\User\test.svg =>
```

```
❶ 3f5794c522e83b827054183658ce63cb701dc49f4e59335f08b5c79c56873969
```

```
C:\Users\User\dev\gnp\ch06>sha512-256sum tftp\tftp\payload.svg
```

```
tftp\tftp\payload.svg =>
```

```
❷ 3f5794c522e83b827054183658ce63cb701dc49f4e59335f08b5c79c56873969
```

Listing 6-13: Generating SHA512/256 checksums for test.svg and payload.svg

As you can see, the *test.svg* checksum ❶ is equal to the *payload.svg* checksum ❷.

In this case, the *test.svg* file is an image of a gopher from Egon Elbre's excellent *gophers* repository on GitHub (<https://github.com/egonelbre/gophers/>). If you opened the file in a web browser, you'd see the image in Figure 6-6.

Although you transferred the payload over localhost and don't expect data loss or corruption, the client and server still acknowledged every data packet, ensuring the proper delivery of the payload.

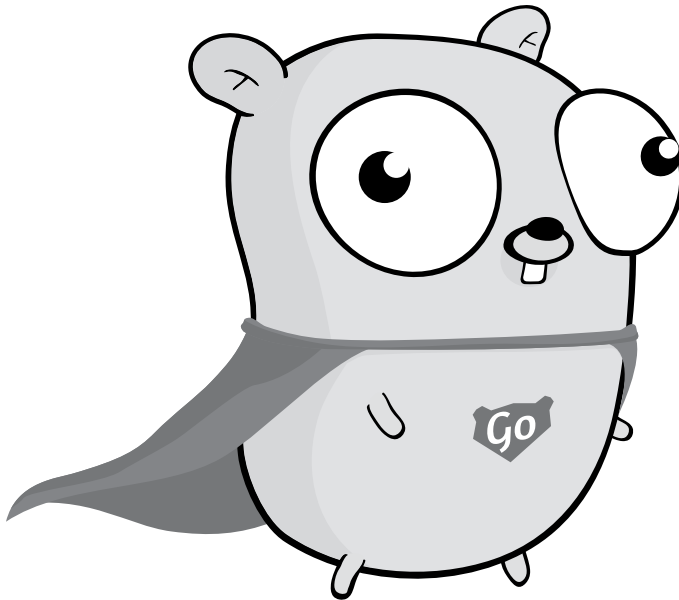


Figure 6-6: Downloaded payload from the TFTP server